Test Document

for

Travelly

Version 1.0

Prepared by

Group 17: **Group Name: 10 Bit**

Ayush Jha	**230258**	**ayushj23@iitk.ac.in**
Khushi Khandelwal	**230561**	**khushik23@iitk.ac.in**
Pranjali Singh	**220796**	**pranjalis22@iitk.ac.in**
Mohd Sarim Ali	**230657**	**sarimali23@iitk.ac.in**
Raghav Madan	**220853**	**raghavm22@iitk.ac.in**
Rigved Koli	**230855**	**brigved23@iitk.ac.in**
Ronit Kumar	**230875**	**ronitk23@iitk.ac.in**
Rushabh Pandya	**230732**	**rushabhbp23@iitk.ac.in**
Sanket Agarwal	**230920**	**sanketag23@iitk.ac.in**
Tanmay Roy	**231082**	**tanmayroy23@iitk.ac.in**

Course: **CS253**

Mentor TA: ***V Arvind***

Date: **03/04/2026**

Contents

Contents 1
Revisions 2

1 Introduction 3

2 Unit Testing 4

3 Integration Testing 32

4 System Testing 40

5 Conclusion 42

Appendix A - Group Log 43

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Full Group	Completed first version of the document	03/04/26

1 Introduction

Test Strategy

Travelly is a travel management application designed to simplify trip planning and management. Our testing strategy mainly involves manual testing, where features are validated by comparing actual results with expected outcomes.

We also use Postman for API testing to ensure backend services function correctly, including data handling and error responses. Functional testing and database validation are performed to maintain reliability and data integrity.

Testing Period of the App

The testing was an integral part of the development process. During the implementation, the app was subjected to tests corresponding to basic functions. However, the majority of the testing of the app was done after the implementation of the app. Since the basic functionality of the app was set in stone and tested, any bugs encountered in the app were easier to tackle since they involved changes only in specific features of the app (certain functions of the source code). This made testing a lot more efficient and less time-consuming.

Testers of the App

Due to time constraints and the size of the team, the app's developers were the testers themselves. However, it was ensured that the responsibility of testing units and components was not placed on the person who developed or played a role in developing them. This ensured the neutrality of the testing process. The benefit of having the developers be the testers was that we knew the different possible inputs and hence were able to carry out extensive testing to ensure the robustness of the application.

Coverage Criteria for Testing

Testing was primarily guided by the **functional requirements** specified for the application. Each feature was evaluated to ensure it met the intended behavior and user expectations, covering both normal and edge-case scenarios.

Tools Used in Testing

The following tools and methods were used during testing:

- Manual testing for validating UI and feature behavior
- API testing tools (e.g., Postman) for backend verification
- Database inspection for validating data correctness and consistency

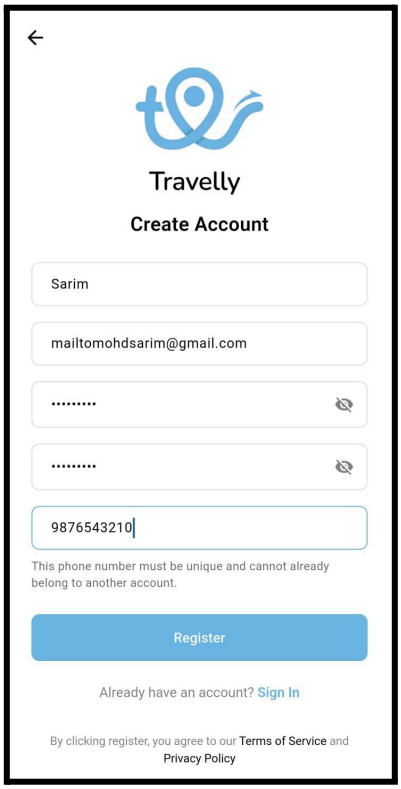
2 Unit Testing

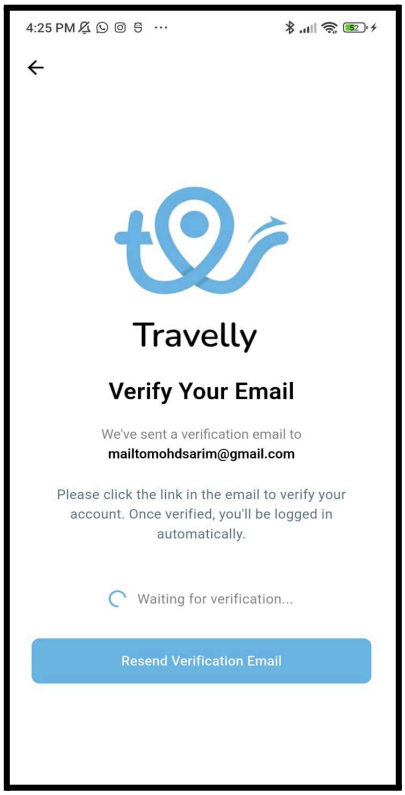
2.1 Authentication

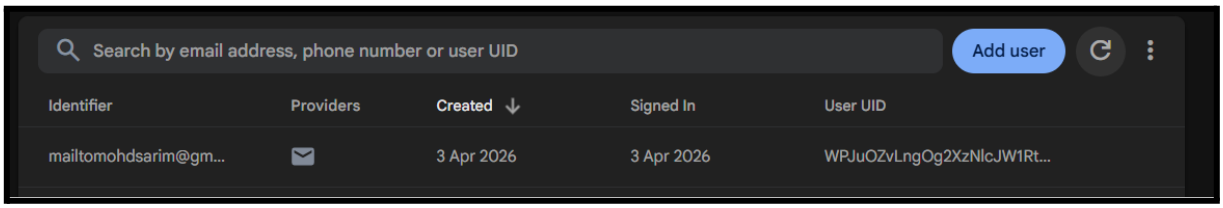
2.1.1 Register User

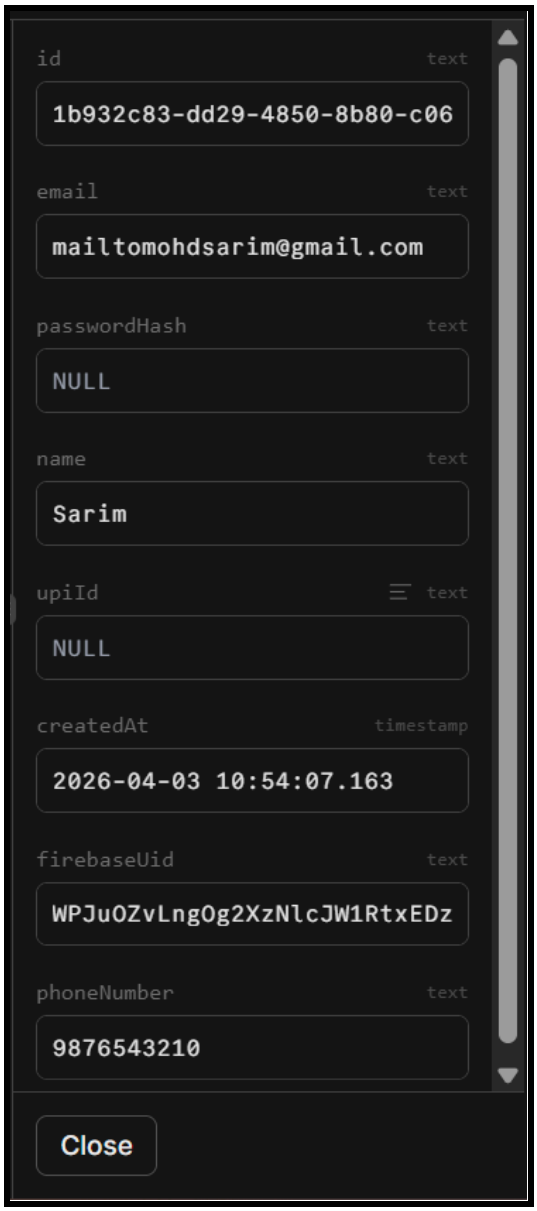
Test Details: Ensures that registering a new user creates a corresponding user entry in the Database.

Test Owner: Sarim
Test Date: 3/4/2026
Test Results: User successfully registered and entry recorded in DB.
Structural Coverage: Functional coverage of Firebase registration flow. Branch coverage for existing vs. new email handling logic.



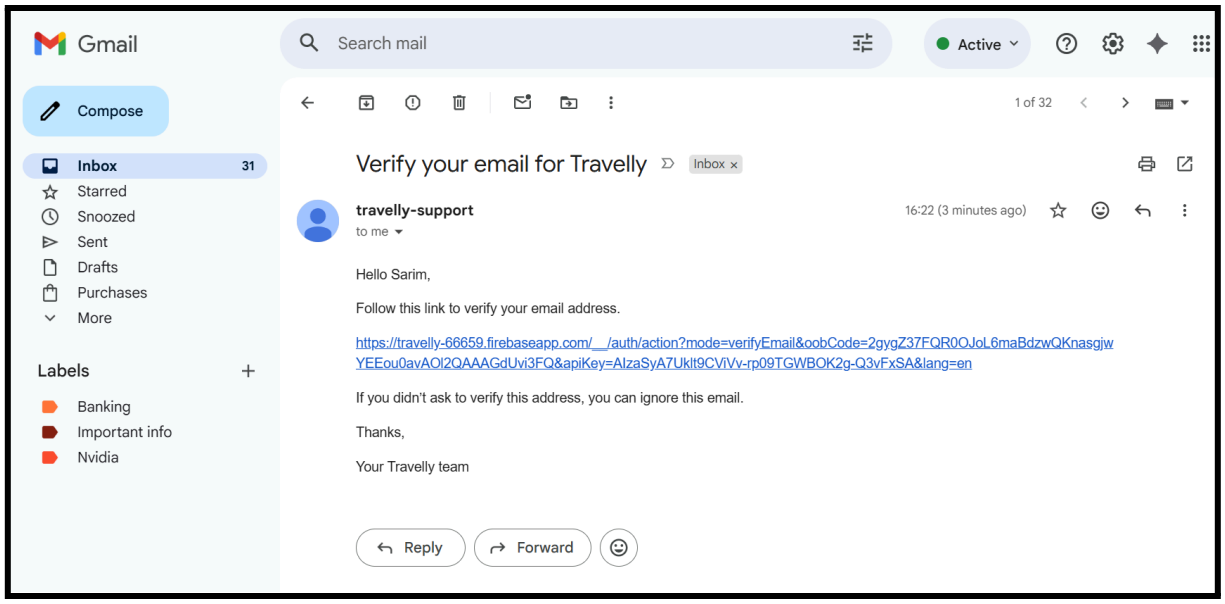






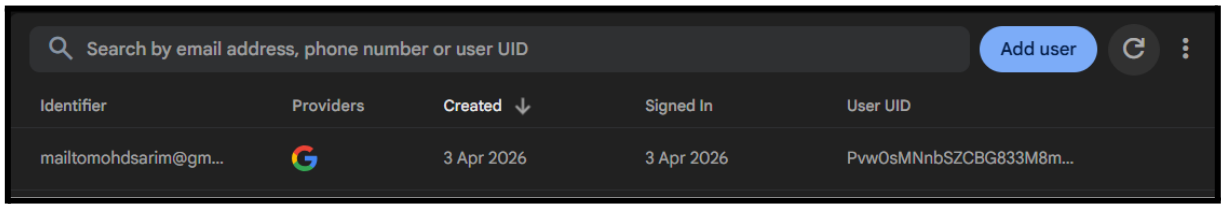
2.1.2 Email Verification

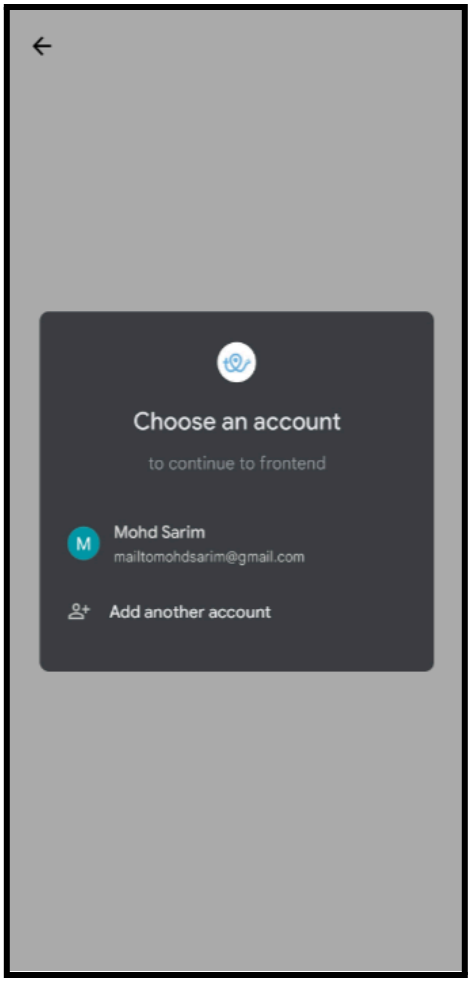
Test Details: Verifies if the verification email is successfully received by the user upon registration.
Test Owner: Sarim
Test Date: 4/3/2026
Test Results: Email received successfully and verified.
Structural Coverage: Functional coverage of the email trigger. Branch coverage on verifying the code correctly limits unverified access.

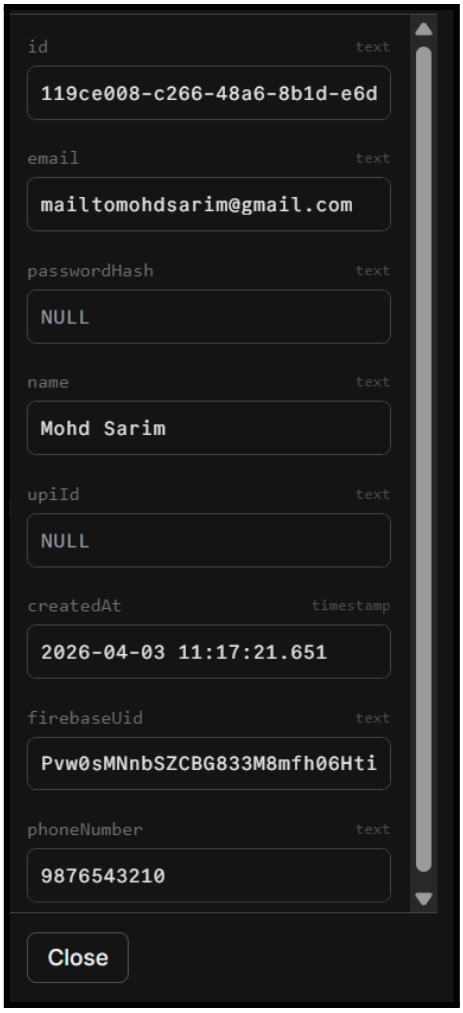


2.1.3 Google Sign-In

Test Details: Checks if Google Sign-In works seamlessly and if an account gets created properly on Firebase.
Test Owner: Sarim
Test Date: 3/4/2026
Test Results: Google authentication successful and Firebase entry verified.
Structural Coverage: Functional coverage of Firebase OAuth integrations and token syncing to NeonDB.



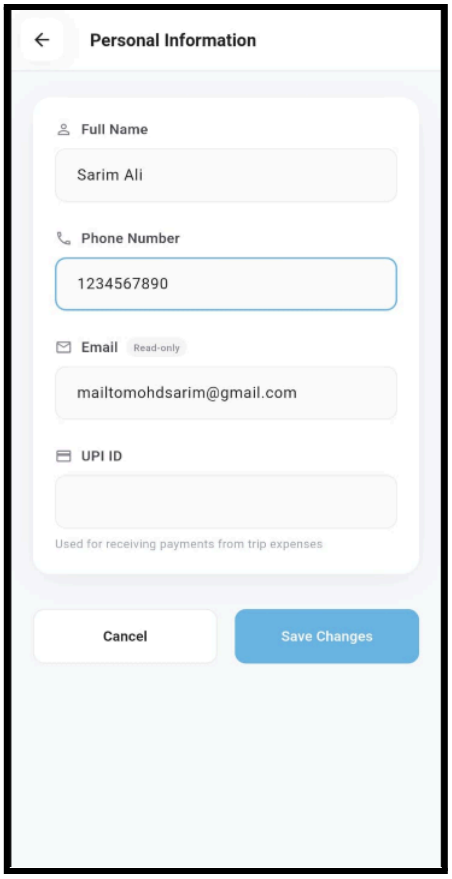


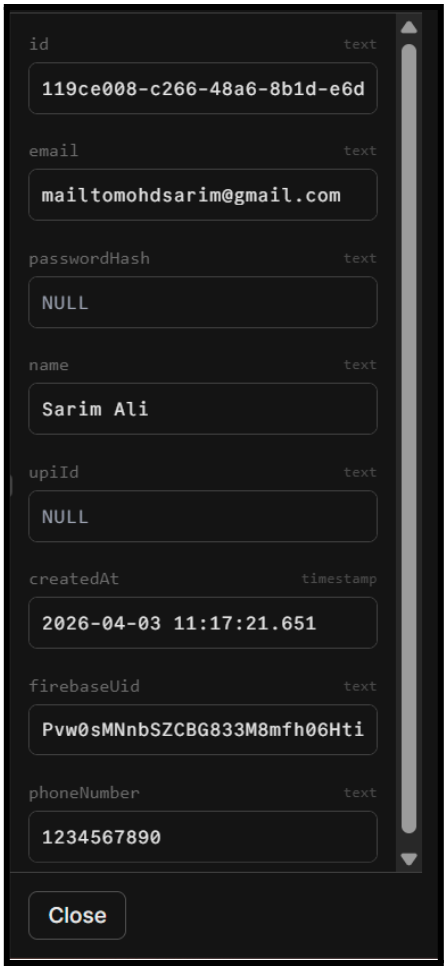


2.2 User Profile

2.2.1 Update Profile Details

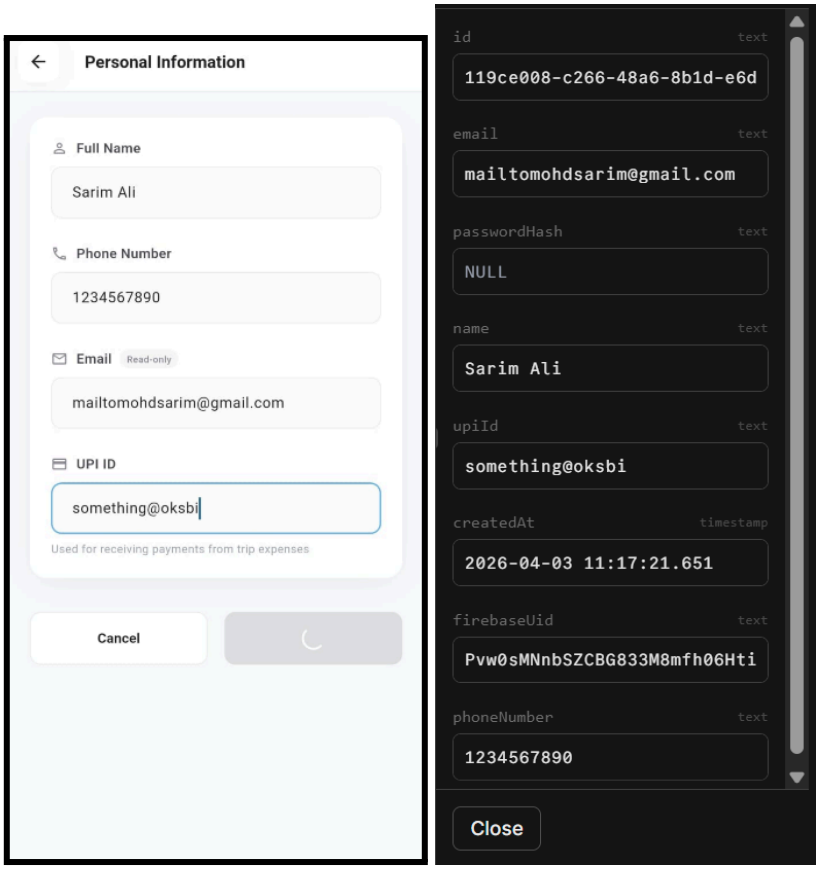
Test Details: Checks whether updating name, email, or phone number correctly reflects in the database.
Test Owner: Ronit
Test Date: 4/3/2026
Test Results: Database fields updated accurately with new information.
Structural Coverage: Functional coverage covering successful profile updates.





2.2.2 Save UPI ID

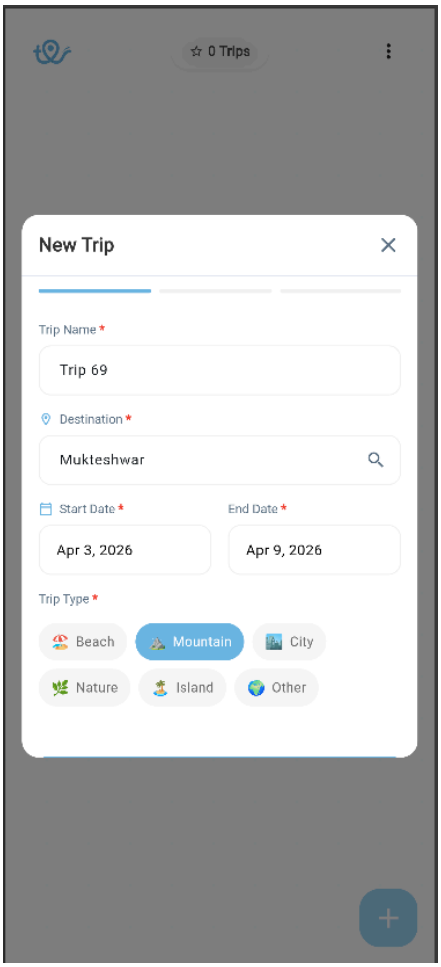
Test Details: Verifies that a valid UPI ID input is stored correctly in the database.
Test Owner: Ayush Jha
Test Date: 3/4/2026
Test Results: UPI ID correctly formatted and saved.
Structural Coverage: Branch coverage evaluating correct regex validations for UPI string format Versus invalid text.



2.3 Group Management

2.3.1 Create Group

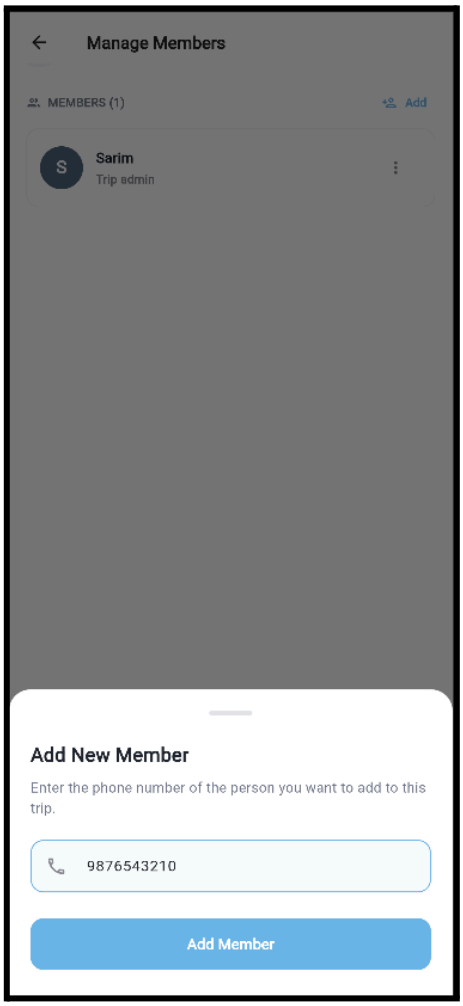
Test Details: Ensures that creating a group triggers a group entry creation in the database.
Test Owner: Pranjali
Test Date: 26/03/2026
Test Results: Group inserted into DB successfully.
Structural Coverage: Functional coverage over group initiation. Statement coverage over missing mandatory fields vs full payload

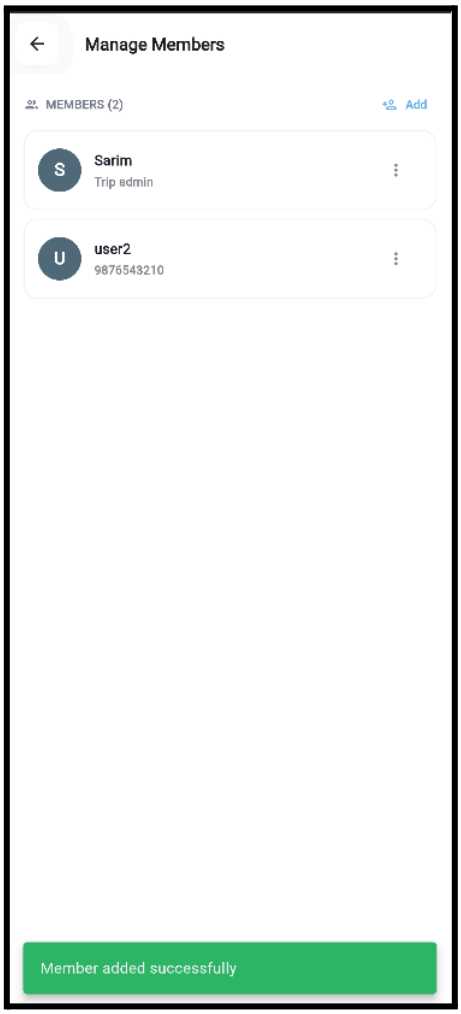

☆ 0 Trips
New Trip
Trip Name *
Trip 69
Destination *
Mukteshwar
Start Date *
End Date *
Apr 3, 2026
Apr 9, 2026
Trip Type *
Beach
Mountain
City
Nature
Island
Other

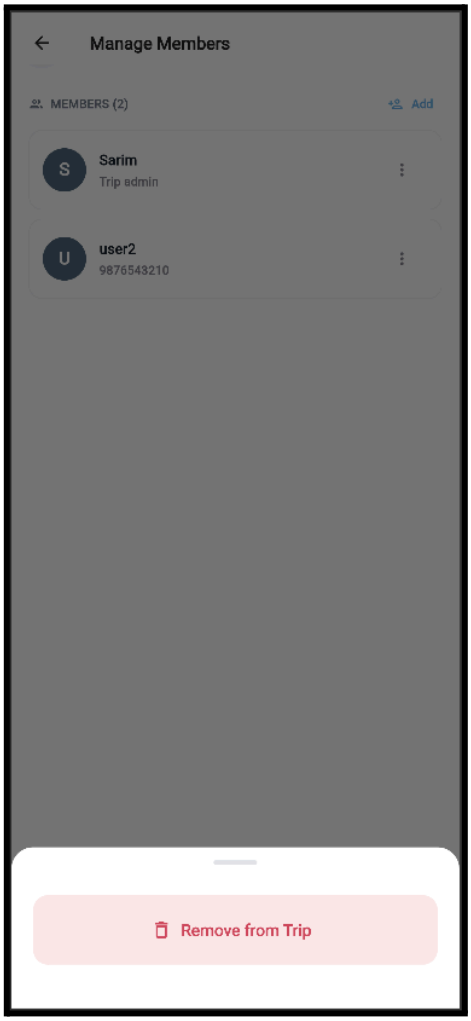

id text
c7fa3682-5dc9-4544-a4de-f0d
title text
Trip 69
inviteLink text
invite-1775233167231-uyvth1
createdBy text
14f7e095-c3e0-4a89-b681-9d9
currency text
INR
createdAt timestamp
2026-04-03 16:19:27.237
simplifyDebts boolean
FALSE
inviteLinkStatus text
ACTIVE
preAddedParticipants jsonb
1 []
destination text
Mukteshwar
startDate timestamp
2026-04-02 18:30:00
endDate timestamp
2026-04-08 18:30:00
tripType text
Mountain
coverImage text
NULL
photoPath text
NULL

2.3.2 Add/Remove Participant

Test Details: Ensures that a host directly adding/removing a participant correctly updates the database.
Test Owner: Sarim
Test Date: 3/4/2026
Test Results: Participant effectively recorded/deleted in DB.
Structural Coverage: Functional coverage enforcing permissions (only admins/creators can add/remove members).









c7fa3682-5dc9-4544-a4d… | Trip 69 | invite-1775233167231-u… | 14f7e095-c3e0-4a89-b68… | INR | 2026-04-03 16

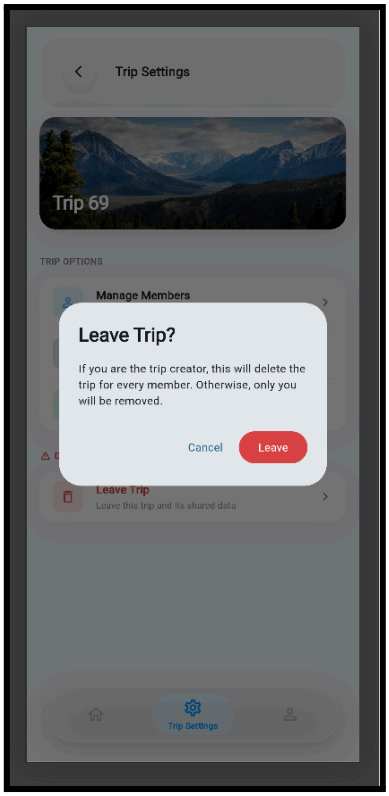
GroupMember

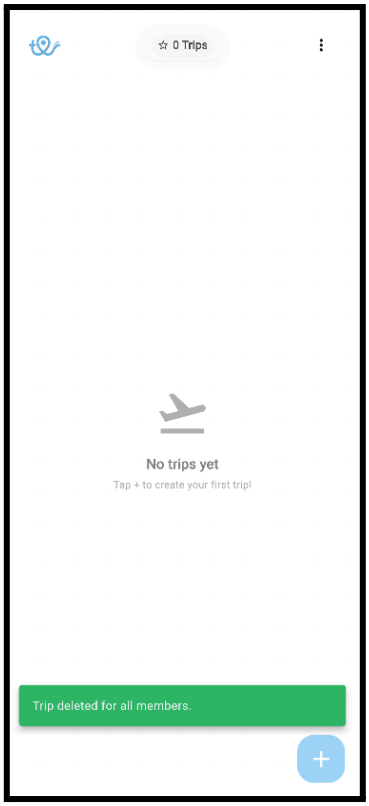
id text	userId text	groupId text	joinedAt timestamp
e021df95-a36e-43b2-8a37-4932…	14f7e095-c3e0-4a89-b681-9d94…	c7fa3682-5dc9-4544-a4de-f0d7…	2026-04-03 16:19:27.914
ab9c075f-65b8-44b4-9493-6384…	230e3e91-f403-4f24-a10e-3f83…	c7fa3682-5dc9-4544-a4de-f0d7…	2026-04-03 16:28:36.37



2.3.3 Delete Group

Test Details: Verifies that deleting a group completely removes the relevant records from the database and invalidates associated links.
Test Owner: Sanket
Test Date: 2/4/2026
Test Results: Group record eliminated natively from DB.
Structural Coverage: Functional structural coverage verifying cascading deletions remove children records without orphaned entries.



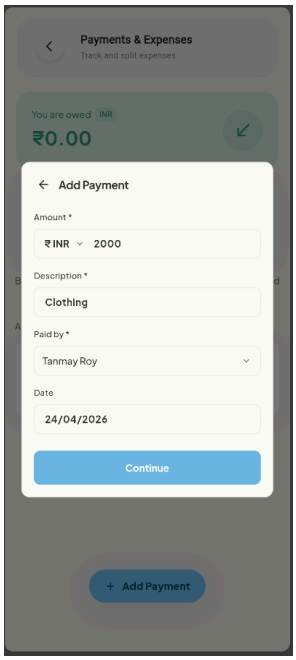


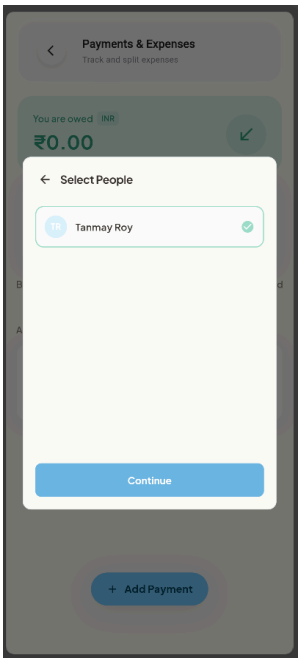
2.4 Expense Management

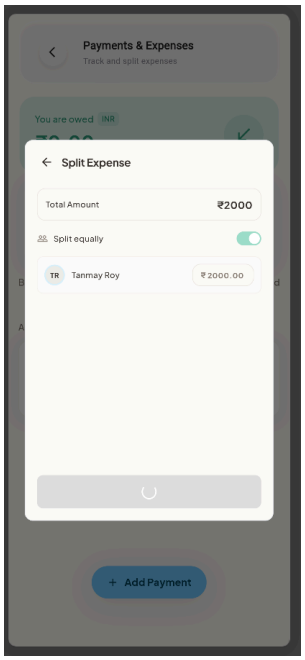
2.4.1 Add Expense

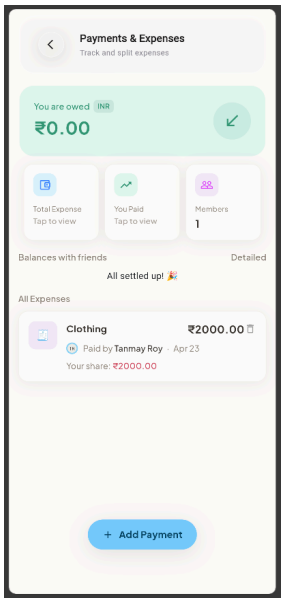
Test Details: Verifies that submitting an expense saves a definitive record in the DB.

Test Owner: Tanmay
Test Date: 29/03/2026
Test Results: Core expense stored precisely.
Structural Coverage: Functional coverage of basic expense submission.





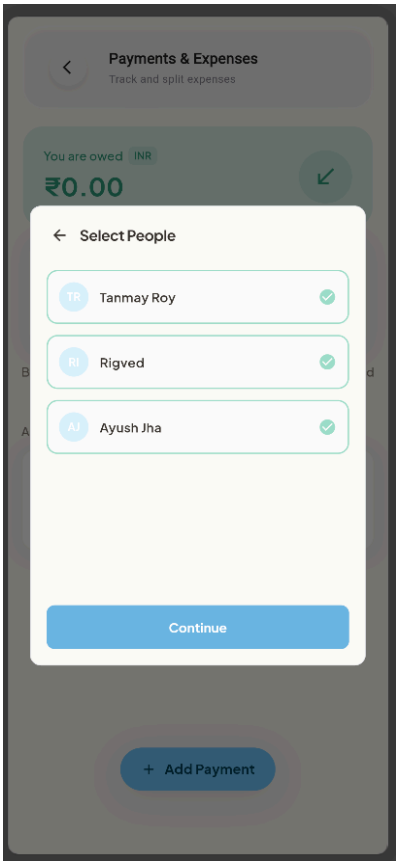


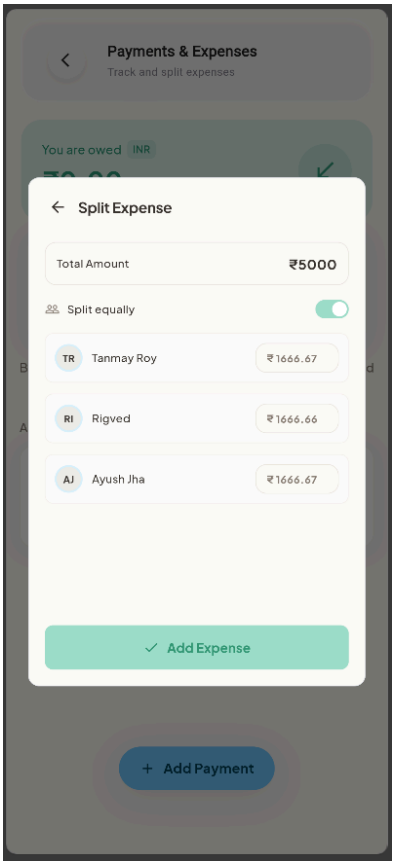


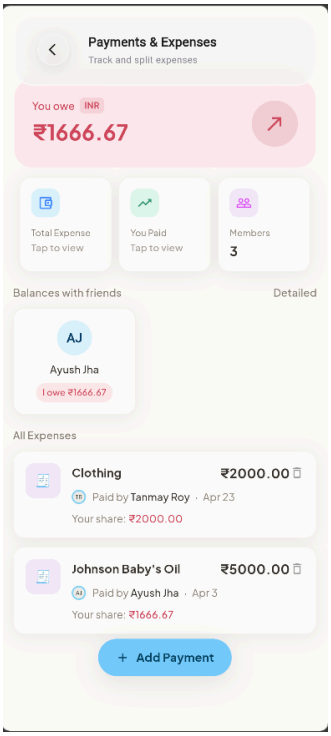


2.4.2 Equal Split

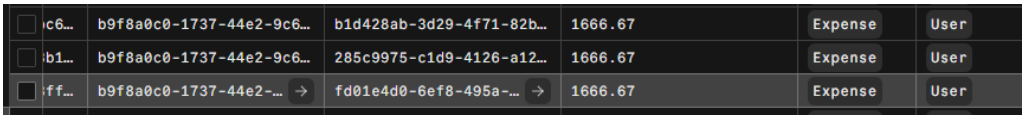
Test Details: Validates that selecting an equal split properly distributes decimal calculations and stores correct shares in the DB.
Test Owner: Tanmay
Test Date:29/03/2026
Test Results: Calculation logic verified against DB states.
Structural Coverage: Statement coverage for arithmetic logic, ensuring fractional totals (e.g. 100/3) are handled stably avoiding 0.333333 breaks.





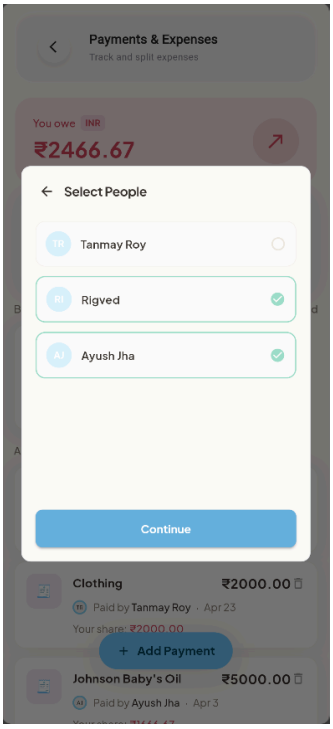


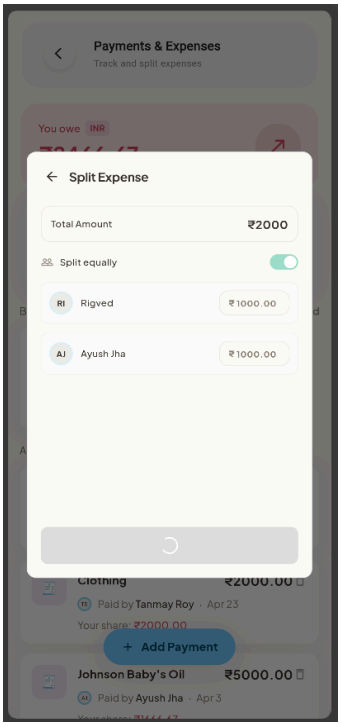


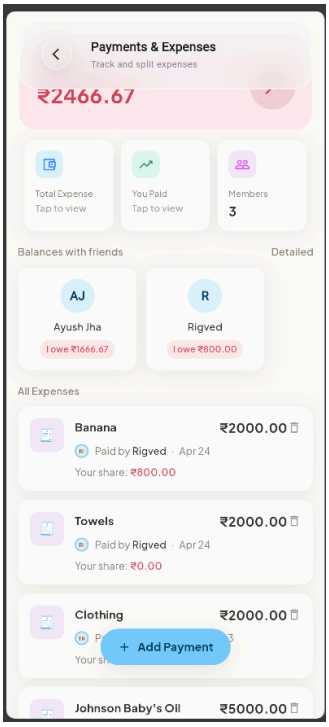


2.4.3 Selective Split

Test Details: Ensures only the selected users are recorded in the split logs and ignores unselected group members.
Test Owner: Tanmay
Test Date: 29/03/26
Test Results: Database entries solely reflect chosen splitters.
Structural Coverage: Decision coverage ensuring ignored members are securely passed over in the distribution loop.









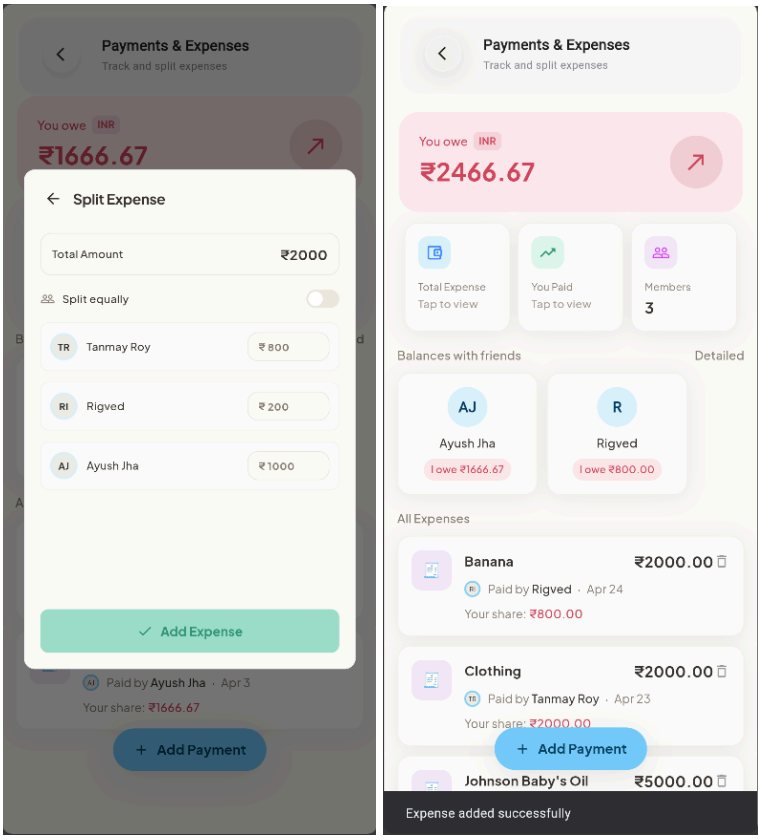


id text	expenseId text	userId text	amount double precision	Expense	User
f43648c7-1f4c-4185-922…	044aa25b-671f-4381-bdc…	53176ea4-ad09-475c-8f7…	1000	Expense	User
043a82a9-7f50-4561-9e4…	044aa25b-671f-4381-bdc…	71c8ad83-b204-4bbe-97a…	1000	Expense	User

2.4.4 Custom Split

Test Details: Proves that an explicitly provided custom proportion is strictly obeyed and saved in the DB.

Test Owner: Khushi
Test Date: 29/03/2026
Test Results: Absolute numeric values saved to DB.
Structural Coverage: Branch coverage capturing validation logic when a custom split's sum fails to identically match the true total expense value.



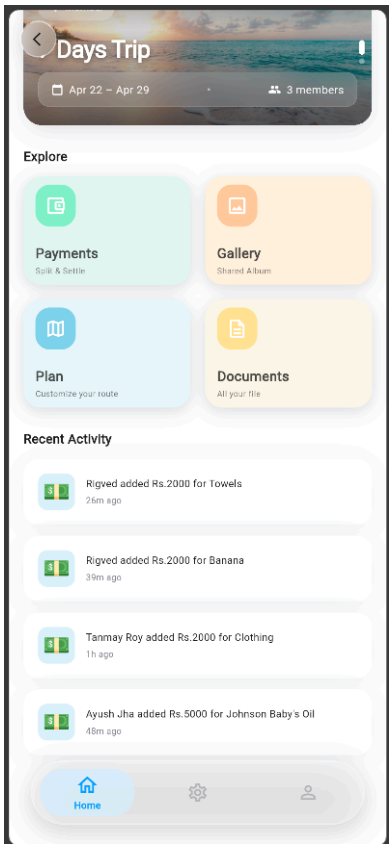


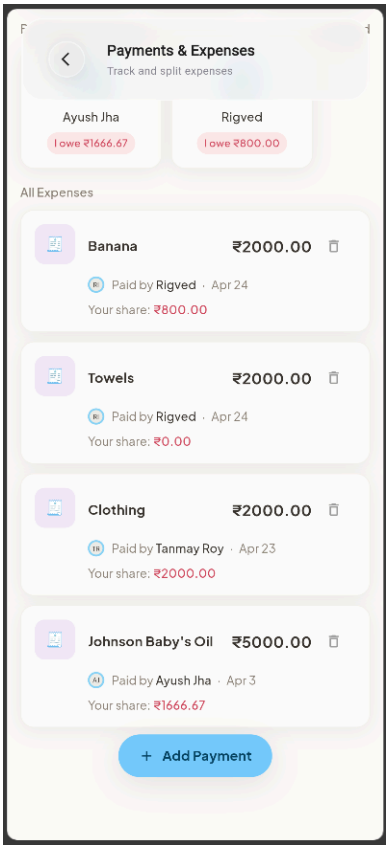
☐	556844ed-6346-408c-a07…	456f49b1-0619-4ad3-b2f…	285c9975-c1d9-4126-a12…	800	Expense	User
☐	c37a226d-ea08-4c36-90e…	456f49b1-0619-4ad3-b2f…	b1d428ab-3d29-4f71-82b…	1000	Expense	User
☐	6b651b2d-64e4-44c1-901…	456f49b1-0619-4ad3-b2f…	fd01e4d0-6ef8-495a-915…	200	Expense	User

2.4.5 Expense History

Test Details: Checks that retrieving the expenses returns chronological data backed correctly by DB timestamps.
Test Owner: Sarim
Test Date: 26/03/26
Test Results: Temporal retrieval matches DB ordering perfectly.
Structural Coverage: Functional coverage ensuring pagination and sorting logic relies reliably on

`.createdAt` attributes.

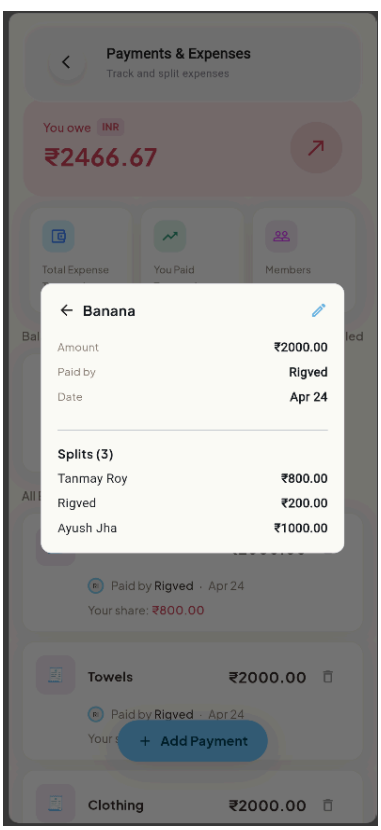


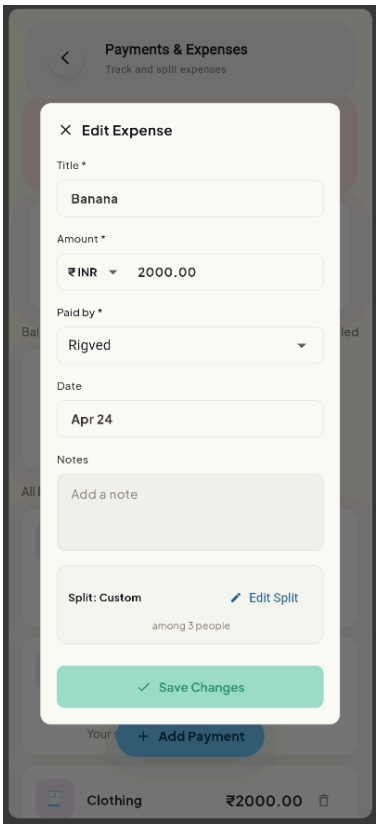


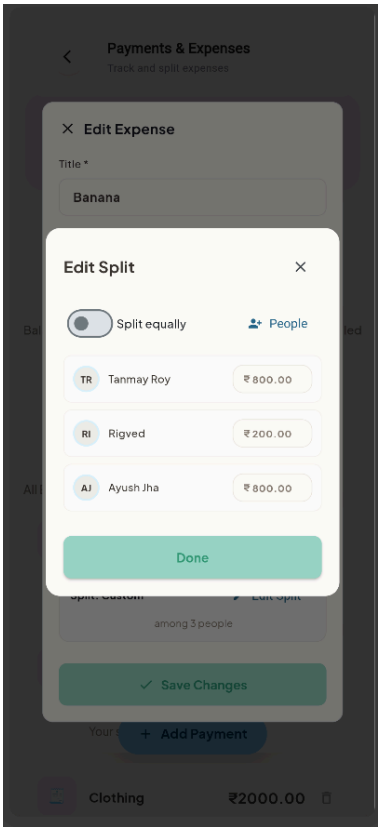
8b1ad185-4c32-478b-81a…	Towels	2000	INR	532dd81a-1f75-4ea3-977…	fd01e4d0-6ef8-495a-915…	2026-04-24 18:30:00	NULL	EQUAL
b9f8a0c0-1737-44e2-9c6…	Johnson Baby's Oil	5000	INR	532dd81a-1f75-4ea3-977…	b1d428ab-3d29-4f71-82b…	2026-04-03 12:30:18.189	NULL	EQUAL
265af2d9-5b88-4cbd-8fc…	Clothing	2000	INR	532dd81a-1f75-4ea3-977…	285c9975-c1d9-4126-a12…	2026-04-23 18:30:00	NULL	EQUAL
456f49b1-0619-4ad3-b2f…	Banana	2000	INR	532dd81a-1f75-4ea3-977…	fd01e4d0-6ef8-495a-915…	2026-04-24 18:30:00	NULL	CUSTOM

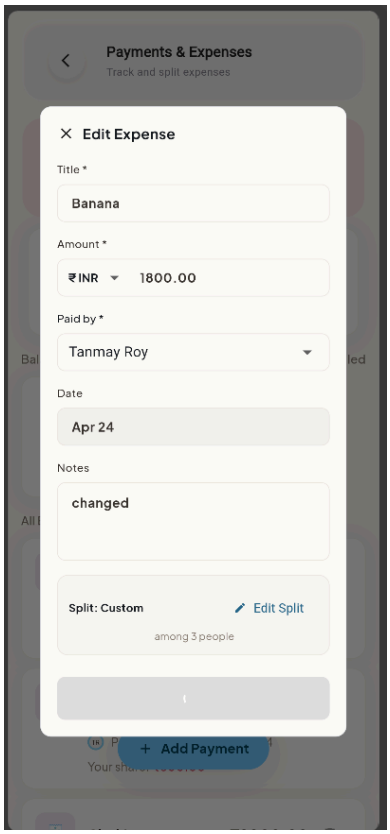
2.4.6 Edit Expense

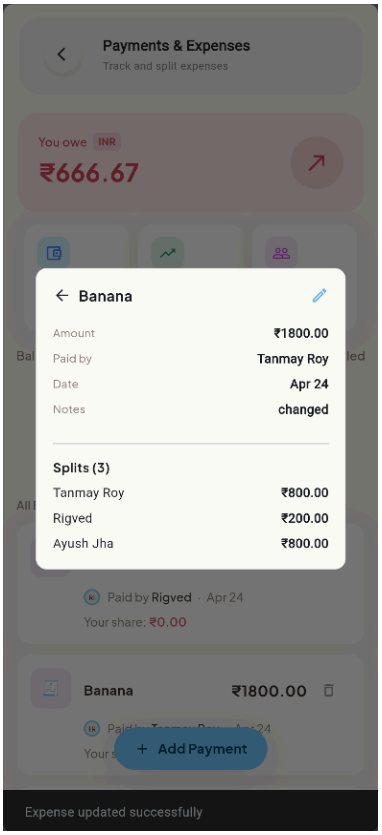
- **Test Details:** Verifies that editing an existing expense correctly updates its details and persists changes in the database, including updating the associated timestamp.
- **Test Owner:** Raghav
- **Test Date:** 26/03/2026
- **Test Results:** Expense updates are successfully saved, and modified timestamps are accurately reflected in the database.
- **Structural Coverage:** Functional coverage ensuring update operations correctly modify records and maintain data consistency across pagination and sorting logic.

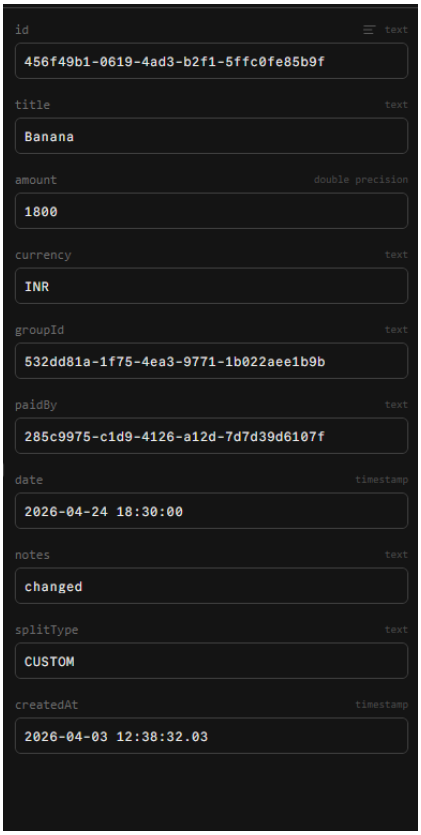

Payments & Expenses
Track and split expenses
You owe INR
₹2466.67
Total Expense
You Paid
Members
Banana
Amount ₹2000.00
Paid by Rigved
Date Apr 24
Splits (3)
Tanmay Roy ₹800.00
Rigved ₹200.00
Ayush Jha ₹1000.00
Paid by Rigved · Apr 24
Your share: ₹800.00
Towels ₹2000.00
Add Payment
Clothing ₹2000.00

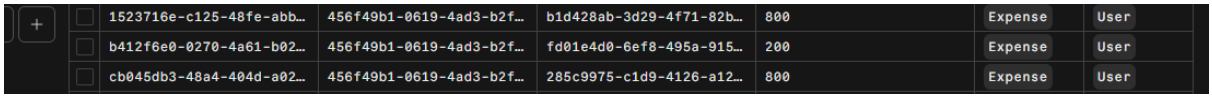

Payments & Expenses
Track and split expenses
Edit Expense
Title *
Banana
Amount *
₹INR 2000.00
Paid by *
Rigved
Date
Apr 24
Notes
Add a note
Split: Custom
Edit Split
among 3 people
Save Changes
Add Payment
Clothing ₹2000.00


Payments & Expenses
Track and split expenses
Edit Expense
Title *
Banana
Edit Split
Split equally
People
TR Tanmay Roy ₹800.00
RI Rigved ₹200.00
AJ Ayush Jha ₹800.00
Done
among 3 people
Save Changes
Add Payment
Clothing ₹2000.00


Payments & Expenses
Track and split expenses
Edit Expense
Title *
Banana
Amount *
₹INR 1800.00
Paid by *
Tanmay Roy
Date
Apr 24
Notes
changed
Split: Custom
Edit Split
among 3 people
Add Payment


Payments & Expenses
Track and split expenses
You owe INR
₹666.67
Banana
Amount ₹1800.00
Paid by Tanmay Roy
Date Apr 24
Notes changed
Splits (3)
Tanmay Roy ₹800.00
Rigved ₹200.00
Ayush Jha ₹800.00
Paid by Rigved · Apr 24
Your share: ₹0.00
Banana ₹1800.00
Add Payment
Expense updated successfully


id text
456f49b1-0619-4ad3-b2f1-5ffc0fe85b9f
title text
Banana
amount double precision
1800
currency text
INR
groupId text
532dd81a-1f75-4ea3-9771-1b022aee1b9b
paidBy text
285c9975-c1d9-4126-a12d-7d7d39d6107f
date timestamp
2026-04-24 18:30:00
notes text
changed
splitType text
CUSTOM
createdAt timestamp
2026-04-03 12:38:32.03



+	1523716e-c125-48fe-abb…	456f49b1-0619-4ad3-b2f…	b1d428ab-3d29-4f71-82b…	800	Expense	User
	b412f6e0-0270-4a61-b02…	456f49b1-0619-4ad3-b2f…	fd01e4d0-6ef8-495a-915…	200	Expense	User
	cb045db3-48a4-404d-a02…	456f49b1-0619-4ad3-b2f…	285c9975-c1d9-4126-a12…	800	Expense	User

2.5 Balance Calculation

2.5.1 Trip Members' Balances

Test Details:
Validates that the Balances endpoint correctly returns a raw breakdown of each member's financial standing within the group. The response should reflect each user's net balance (total paid minus total owed) and be properly organized by currency. Test was done using Postman

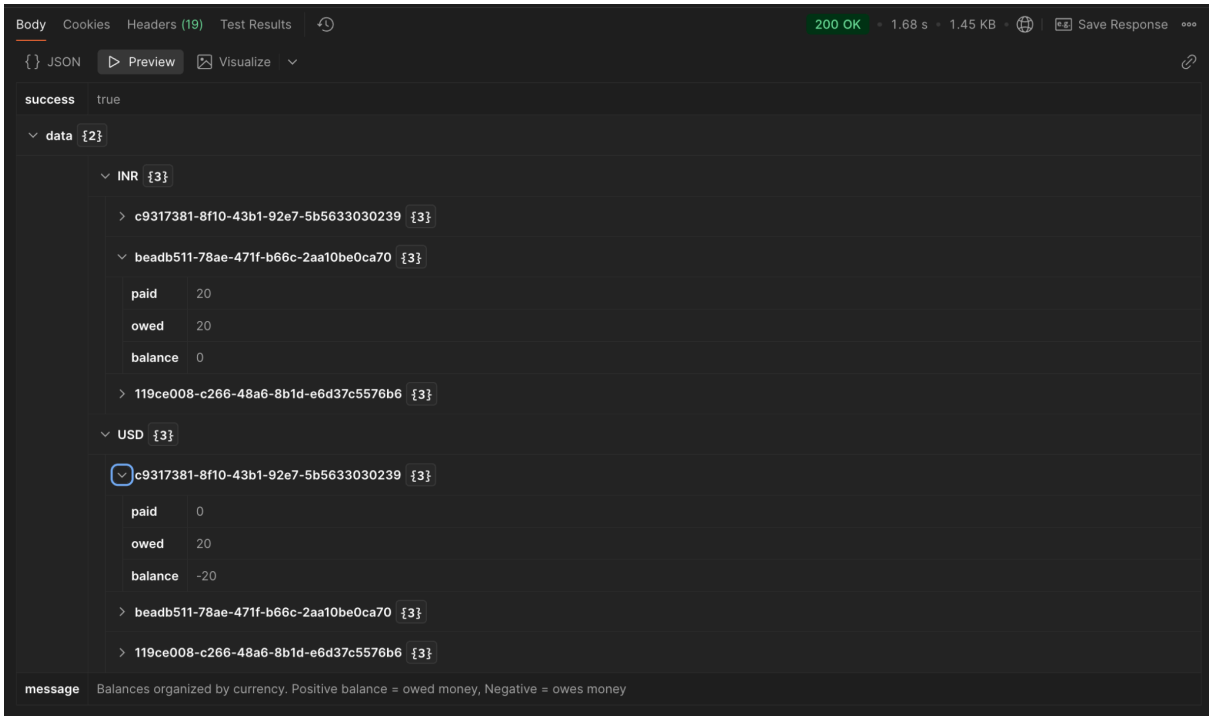
Test Owner: Rigved

Test Date: 29/03/2026

Test Results:
All users' net balances are accurately computed and returned. Values correctly reflect contributions and liabilities across different currencies.

Structural Coverage:
Functional coverage validating backend computation logic for net balances, including aggregation of expenses, split calculations, and currency-wise grouping.





2.5.2 Net Balance Calculation

Test Details: Ensures correct algebraic calculations resulting in the final owed net balance states. Ensures that the simplify-debts logic works. Used Postman for this test

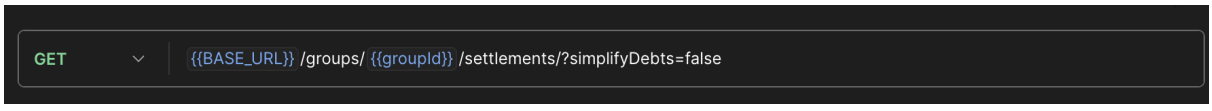
Test Owner: Rigved

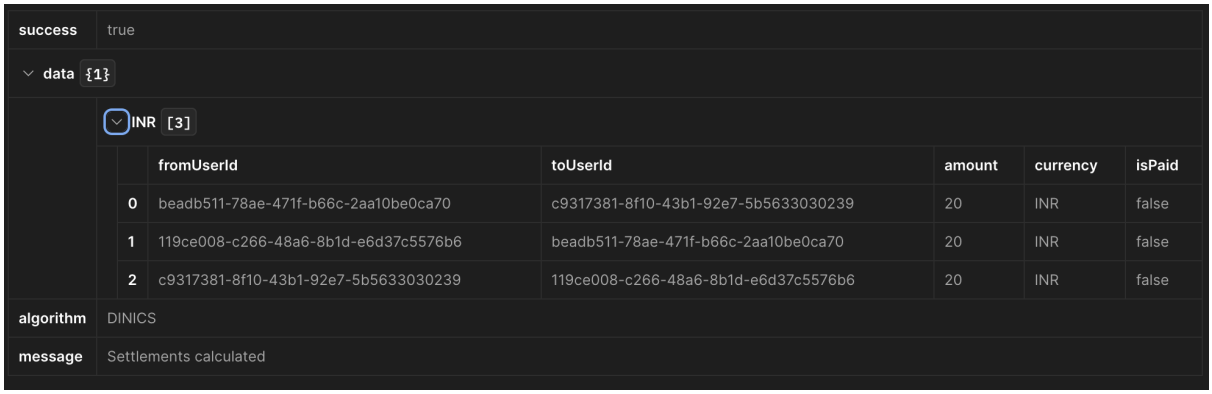
Test Date: 29/03/2026

Test Results: Mathematics of net balances is verified to be logically sound.

Structural Coverage: State coverage testing multiple layered debts resolving cleanly to single simplified sums.

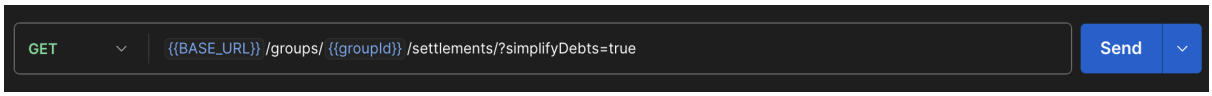
With Simplify Debts = false (raw transactions)

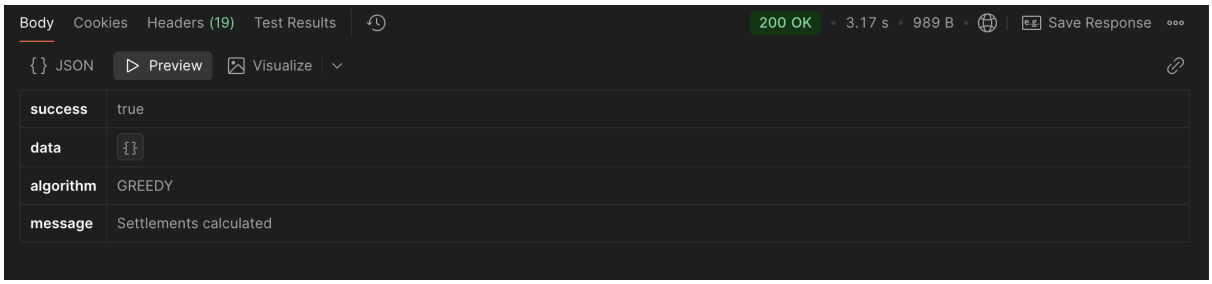




success	true				
data {1}					
INR [3]					
	fromUserId	toUserId	amount	currency	isPaid
0	beadb511-78ae-471f-b66c-2aa10be0ca70	c9317381-8f10-43b1-92e7-5b5633030239	20	INR	false
1	119ce008-c266-48a6-8b1d-e6d37c5576b6	beadb511-78ae-471f-b66c-2aa10be0ca70	20	INR	false
2	c9317381-8f10-43b1-92e7-5b5633030239	119ce008-c266-48a6-8b1d-e6d37c5576b6	20	INR	false
algorithm	DINICS				
message	Settlements calculated				

With Simplify Debts = true (*Number of transactions are minimised as required)*





2.5.3 Simplify Debts Toggle

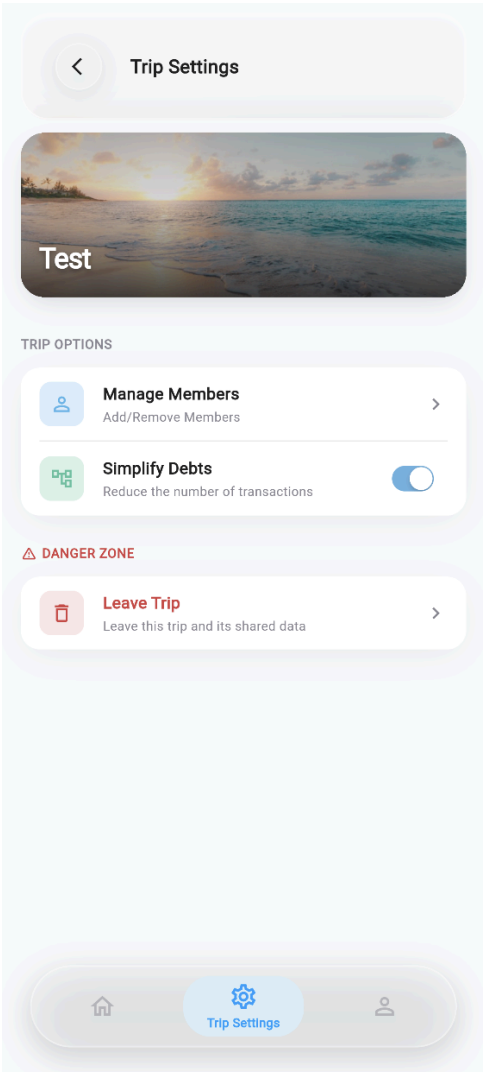
Test Details: Checks whether turning the "simplify debts" setting on/off properly reflects in the database settings matrix.

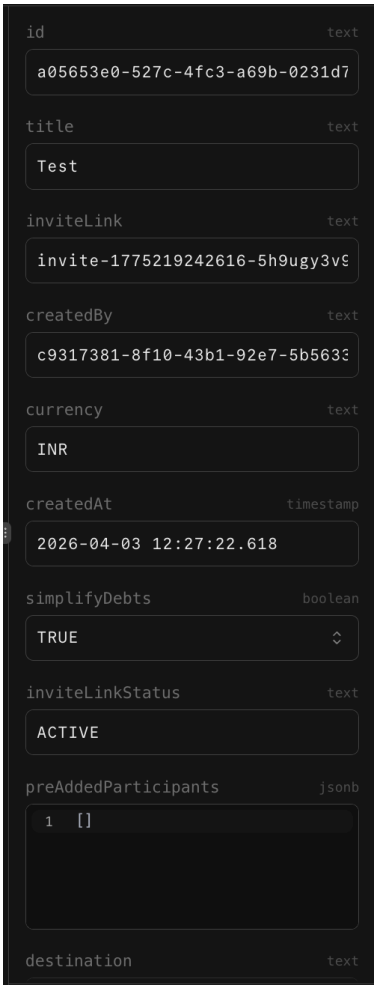
Test Owner: Pranjali

Test Date: 29/03/2026

Test Results: Database toggles boolean value perfectly.

Structural Coverage: Branch coverage verifying debt computation loops act differently based on the boolean state.





2.6 Settlement

2.6.1 Payment Confirmation (Mark as Paid)

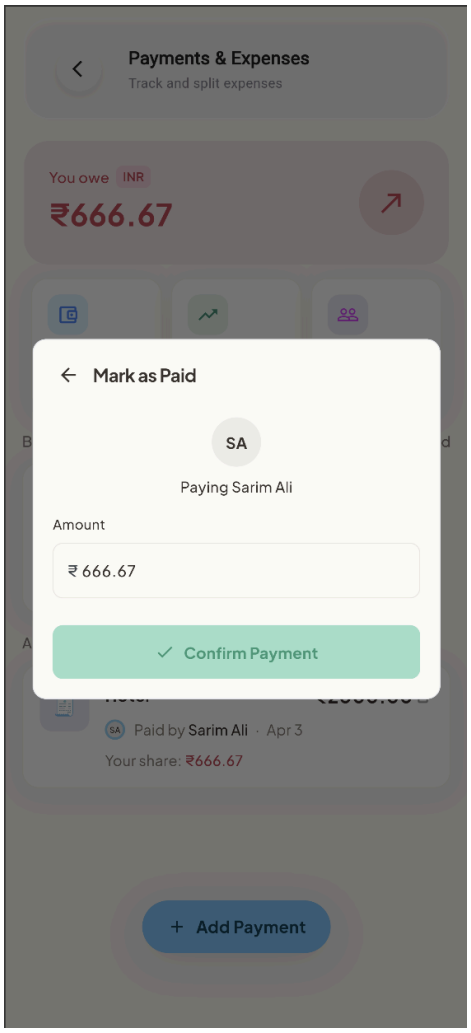
Test Details: Validates that indicating a marked payment correctly clears the ledger balances inside the database.

Test Owner: Ayush

Test Date: 29/03/2026

Test Results: Registers a Transaction and marked complete in DB.

Structural Coverage: Functional coverage of settle payment transaction submission





2.6.2 Pay with UPI (Generates Link)

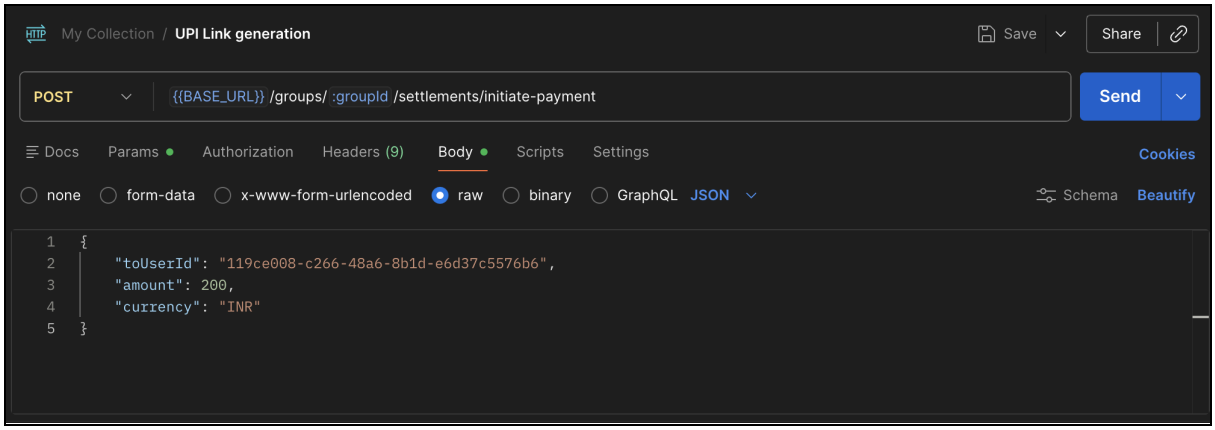
Test Details: Generates contextual URI strings using UPI specs for mobile redirect. Done using Postman

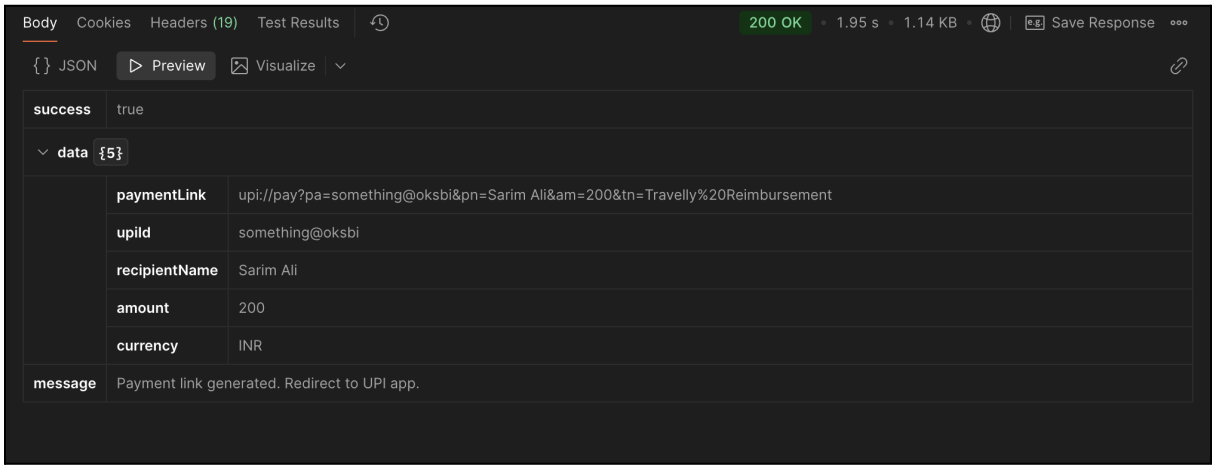
Test Owner: Khushi

Test Date: 29/03/2026

Test Results: Link successfully structured.

Structural Coverage: Functional coverage of URL structuring and link generation

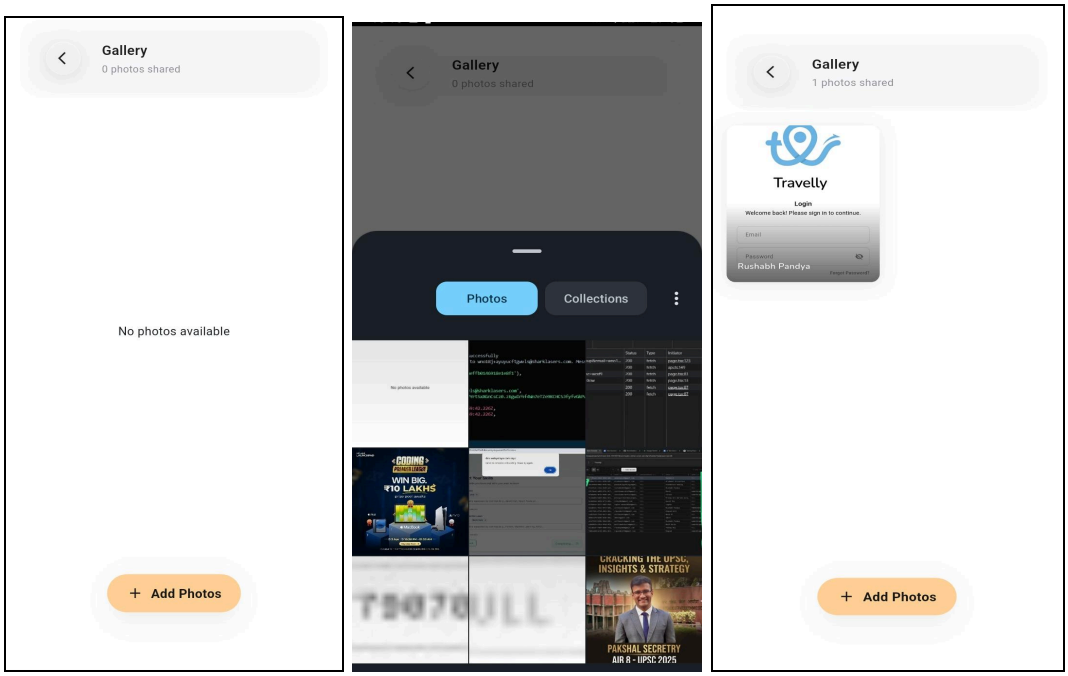




2.7 Photos & Documents

2.7.1 Upload File

Test Details: Checks that sending media payloads results in successful disk/S3 writes alongside saving the file route URL into the DB.
Test Owner: Pranjali
Test Date: 3/4/26
Test Results: File uploaded, DB reference verified.
Structural Coverage: Functional testing handling multipart payload boundary construction routing directly into active backend disks.

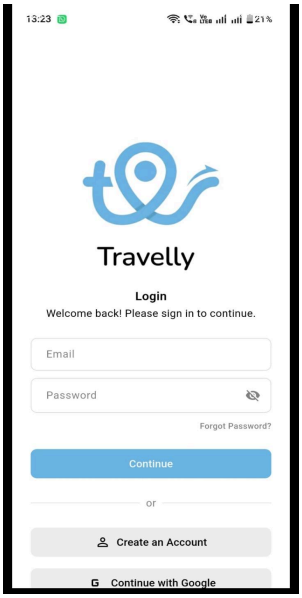




2.7.2 View File

Test Details: Ensures the frontend can render the media using the URI stored in the DB.

Test Owner: Rushabh
Test Date: 3/4/26
Test Results: Image/Document correctly fetched and rendered.
Structural Coverage: Functional display logic.



2.7.3 Delete File

Test Details: Deleting media guarantees that the row in the DB is irrevocably abandoned.
Test Owner: Rushabh
Test Date:3/4/26
Test Results: Validated deletion query removed media from DB.
Structural Coverage: Functional deletion coverage including cleanup scripts purging internal file storage simultaneously.



2.8 Route Optimization

2.8.1 Optimized Route Generation

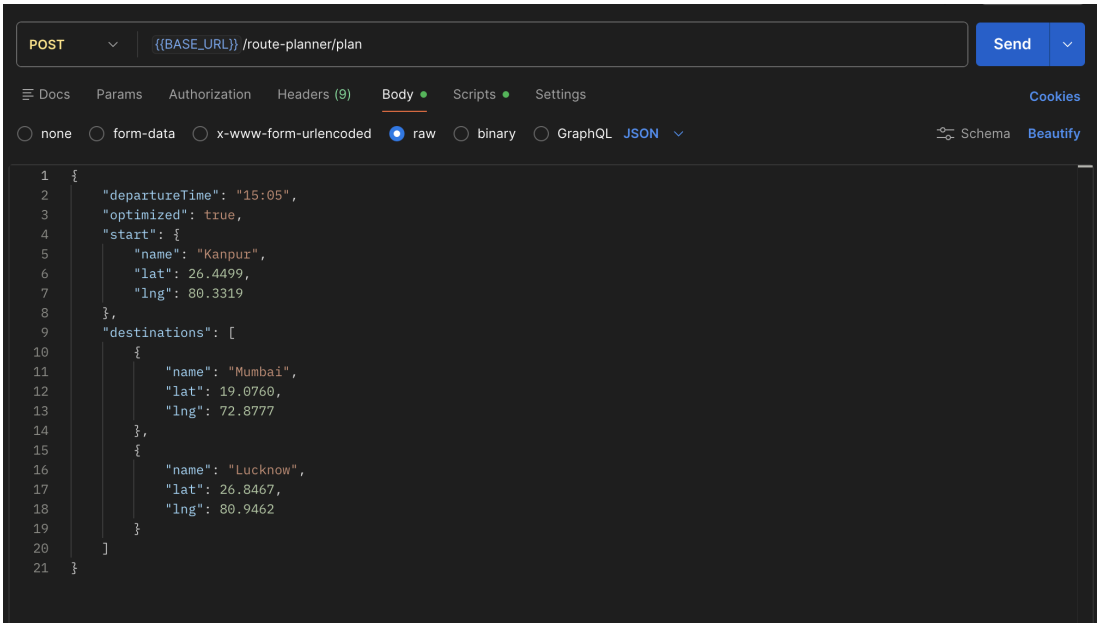
Test Details: Confirms the OpenRouteService spit back chronologically logic routes. Used Postman to send post request

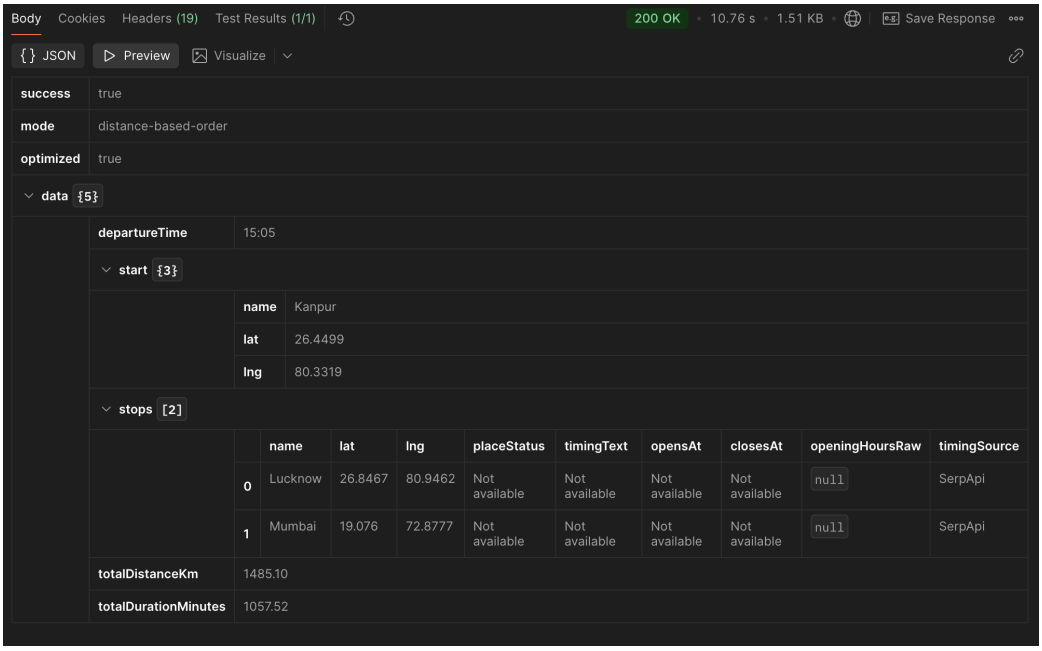
Test Owner: Pranjali

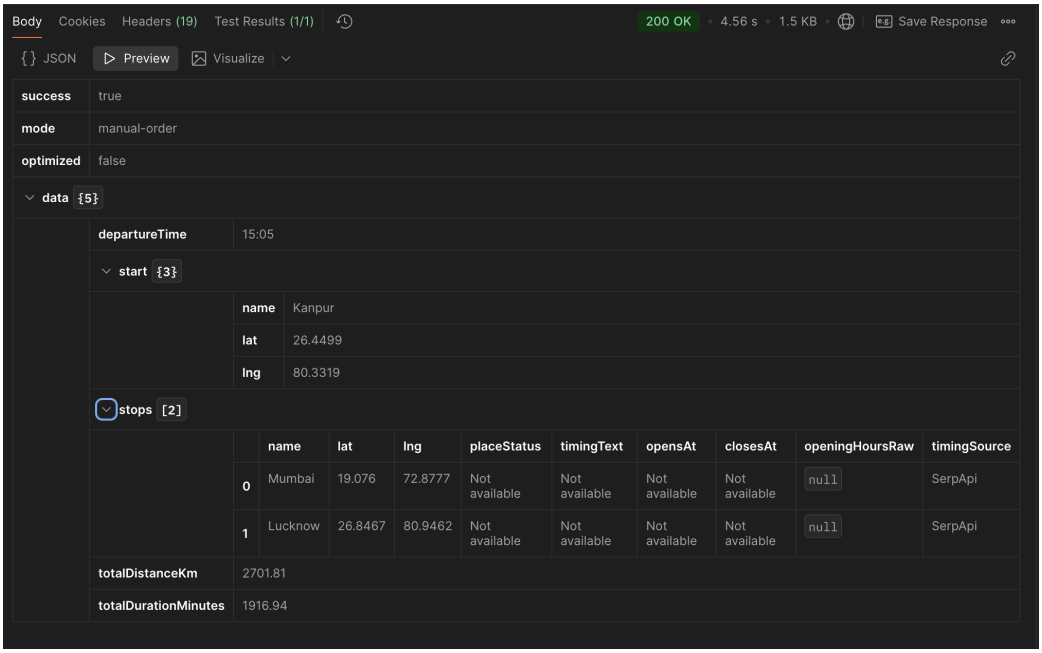
Test Date: 1/04/2026

Test Results: Shortest path accurately fetched.

Structural Coverage: Functional coverage







2.9 Notifications

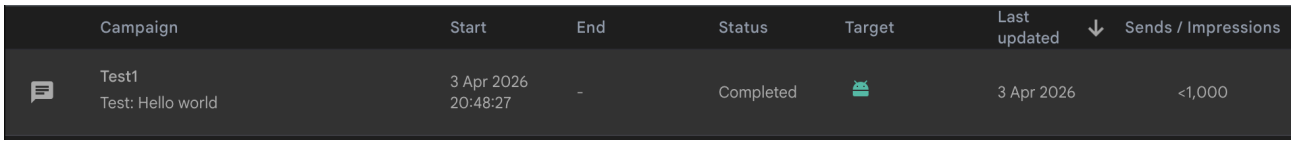
Test Details: Validates push notification delivery and handling using Firebase Cloud Messaging. Test includes sending manual test notifications and verifying automatic notifications triggered on expense addition within the app.

Test Owner: Rigved

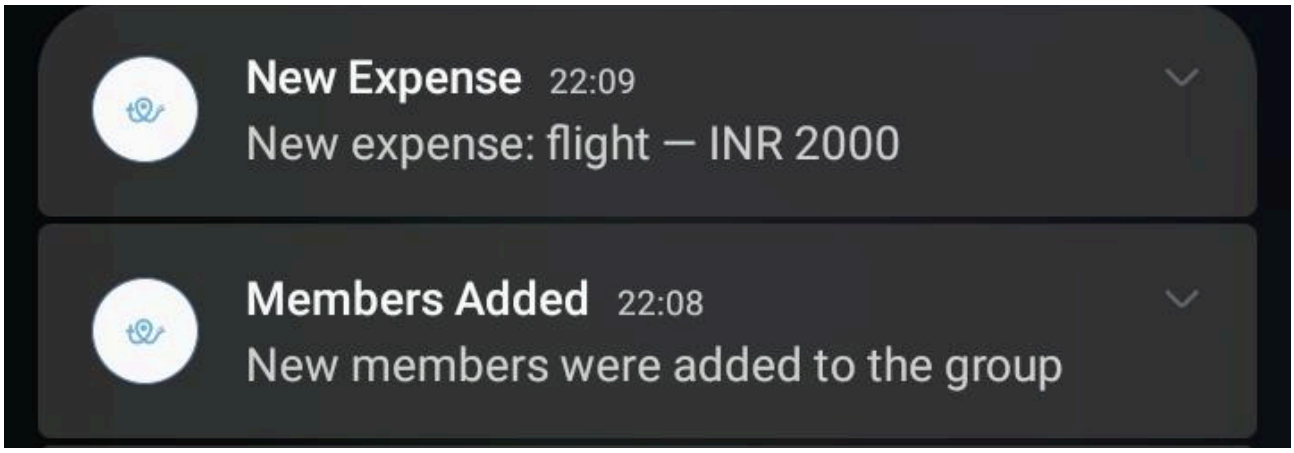
Test Date: 03/04/2026

Test Results: Notifications were successfully delivered and received on target devices. Manual test notifications were received correctly, and automated notifications were triggered and delivered to concerned users upon adding expenses in the app.

Structural Coverage: Functional coverage of notification pipeline including message triggering, cloud delivery, and client-side reception/display.





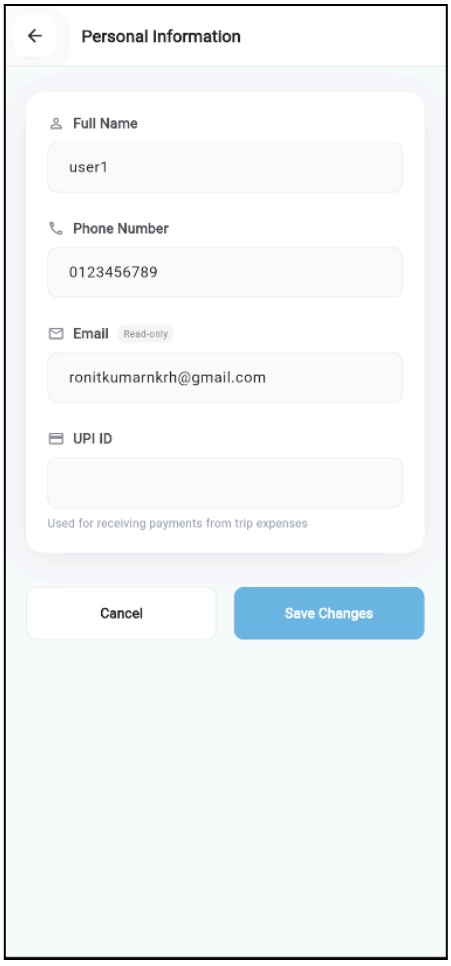


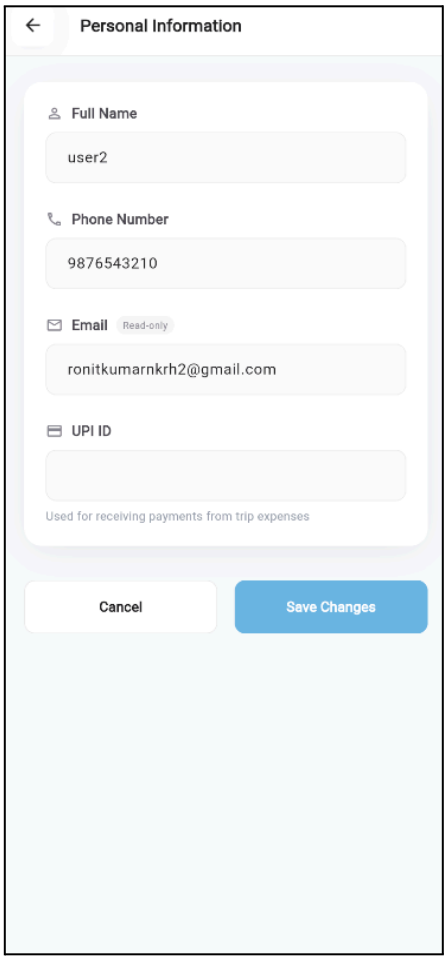
3 Integration Testing

3.1 Trip Creation and Sharing Reflection Across Multiple Users

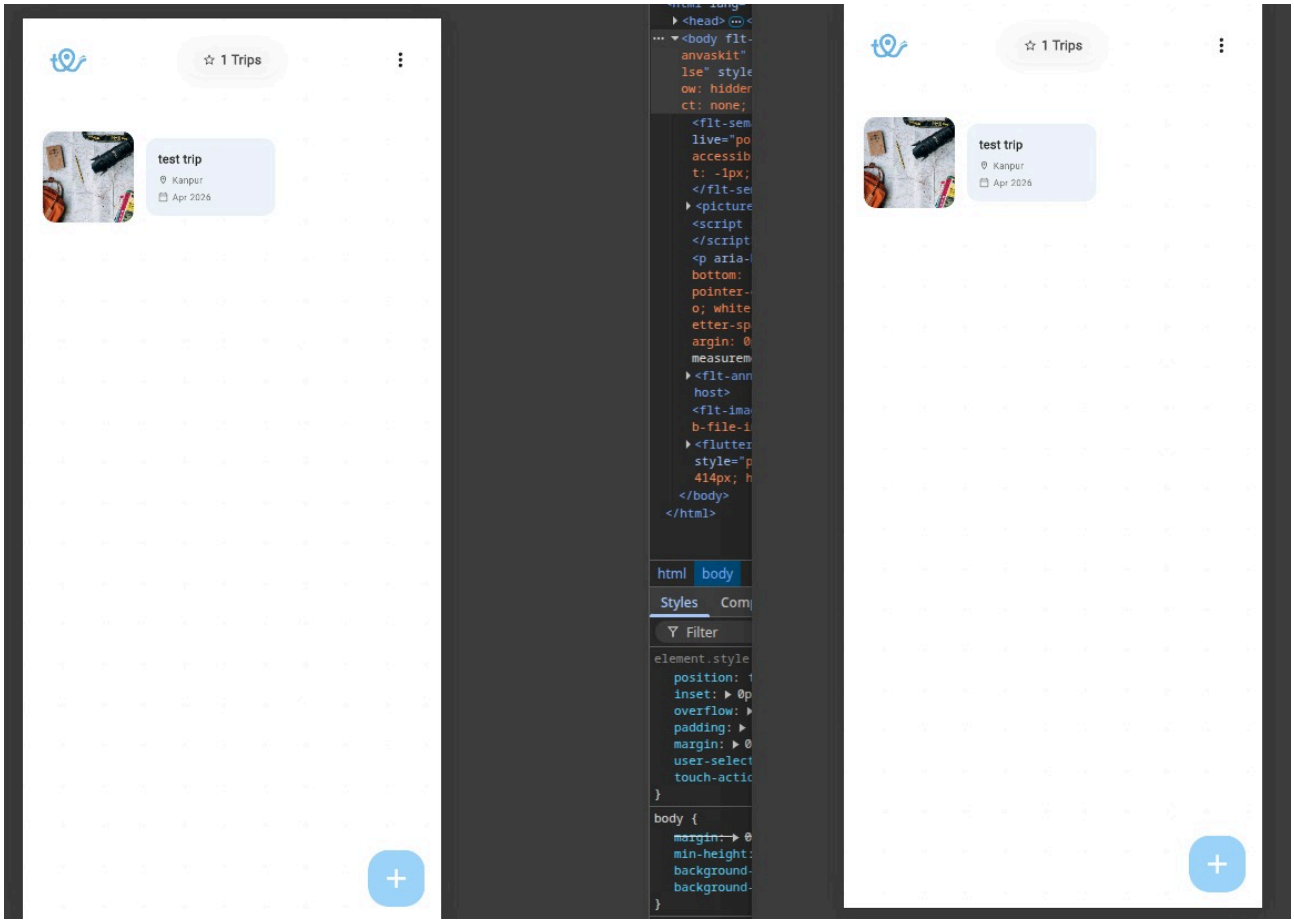
Module Details: This module tests whether a trip created by one user is correctly reflected in another user's account when they are added to the trip.

Test Owner: *Ronit Kumar*
Test Date: *03/04/2026 - 03/04/2026*
Test Results: *Functionality works correctly. Trip created by User 1 is successfully reflected in User 2's account*





Here are two users: User 1 and User 2. User 1 creates a trip/group and adds User 2 to it. The trip should then be visible in User 2's account in its entirety.

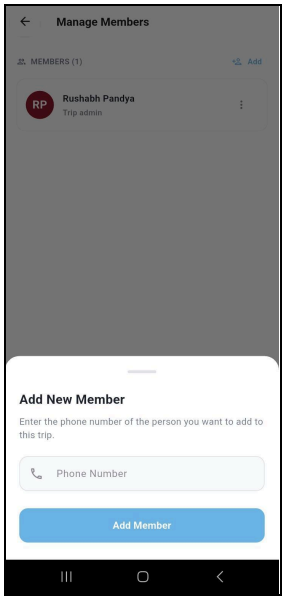


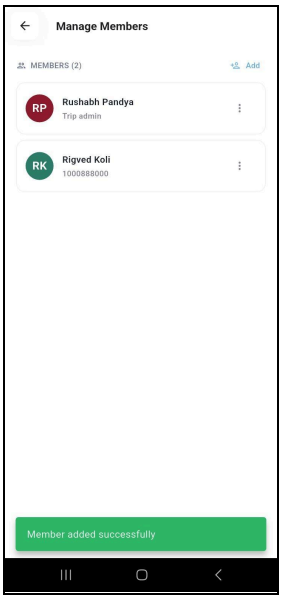
Trip visible on both users

3.2 User ↔ Group

3.2.1 Creator Auto-Added and Member addition

Test Details: User joining the module updates the view context for all other concurrently logged-in members seamlessly. Designing a group mandates that the creator's specific ID is inherently mapped to admin and actively shown everywhere as the primary participant.
Test Owner: Rushabh
Test Date: 3/4/26
Test Results: List updates actively.





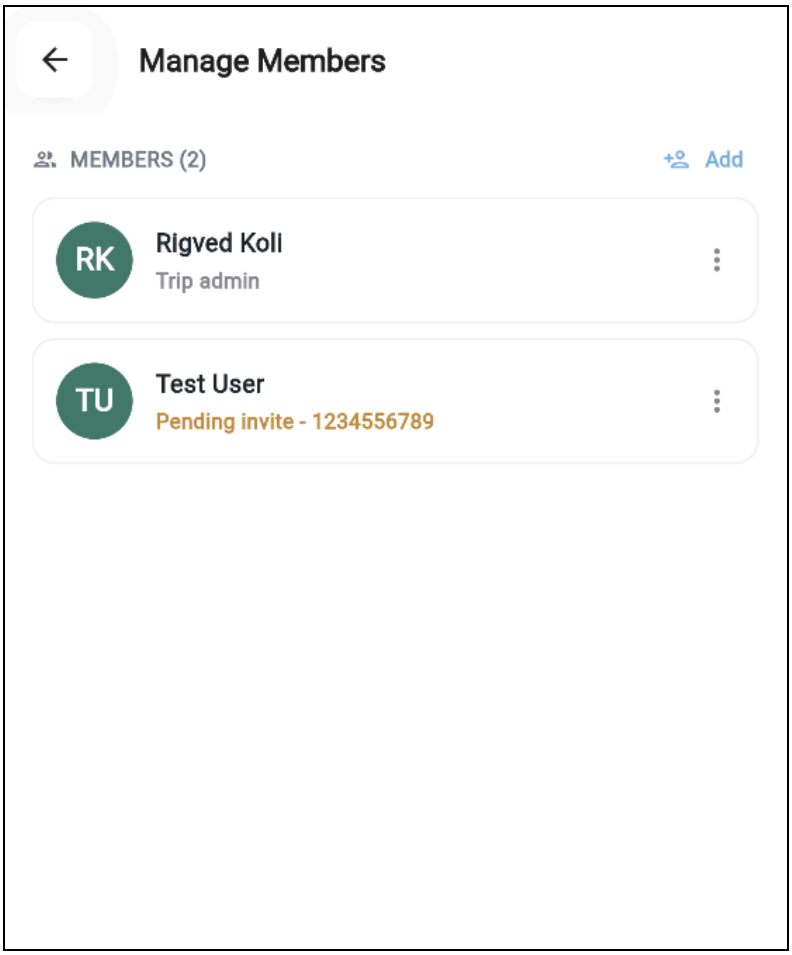
3.2.2 Late Joiner Sync

Test Details: When a participant who has not yet registered on the app is involved in transactions, the system creates a temporary (dummy) user mapped to their phone number. All expenses and payments associated with that number are recorded against this dummy user. Once the actual user registers and is added to the group, all previously recorded transactions are seamlessly mapped and transferred to their account, preserving complete history and balances.

Test Owner: Rigved

Test Date: 01/04/2026

Test Results: Dummy user creation and subsequent data mapping function correctly; all historical transactions are accurately transferred upon user registration and group inclusion.



3.3 Expenses ↔ Group

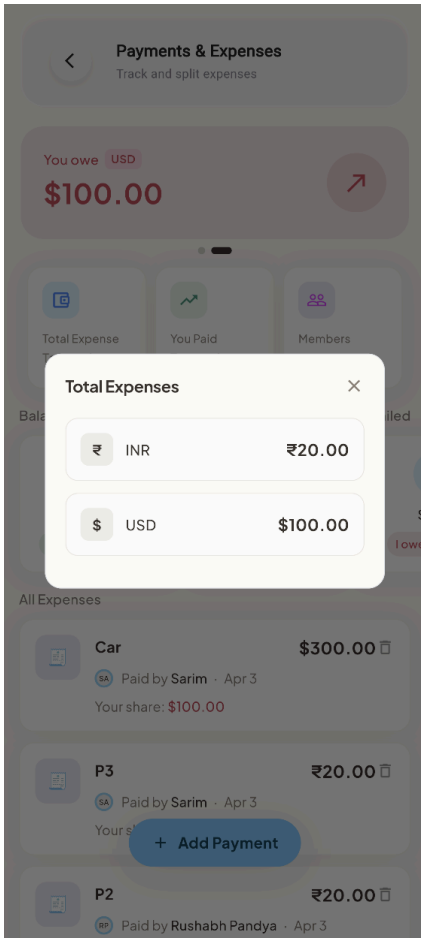
3.3.1 Instant Balance Updates

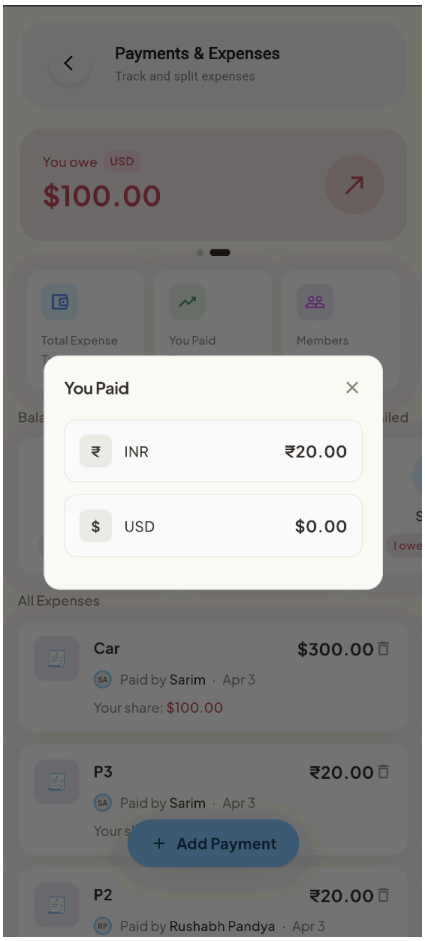
Test Details: The app updates the UI to display the total expenses of the trip along with a detailed breakdown of expenses made by the active user in each currency separately, exactly matching the values returned by the API output.

Test Owner: Sanket

Test Date: 29/03/2026

Test Results: Total and per-currency expense values rendered accurately as per API response; UI updates instantly and reflects correct data.





3.4 Balance ↔ Settlement

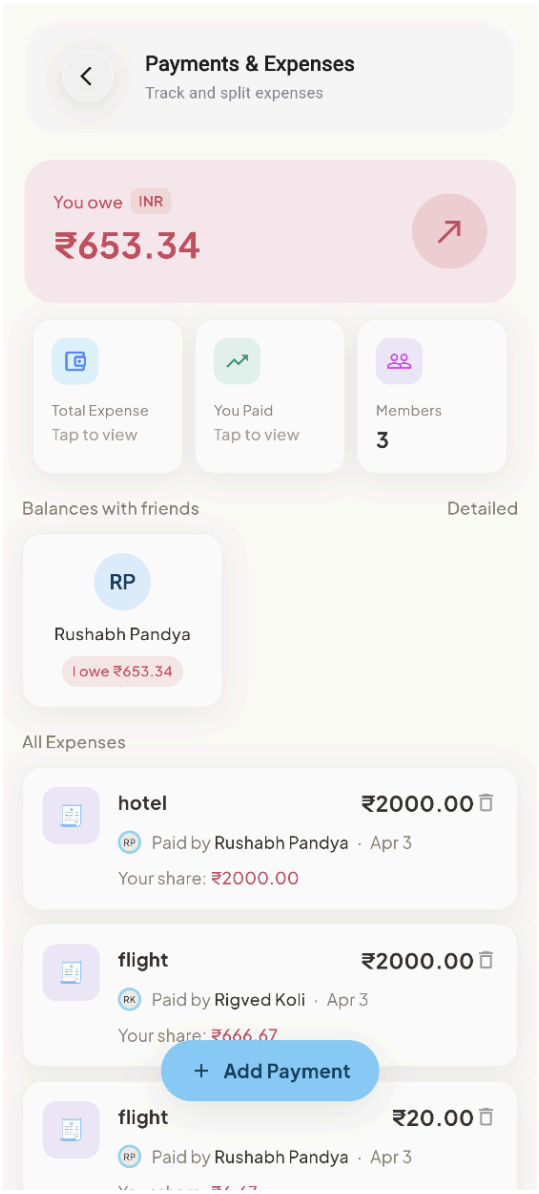
3.4.1 Pending Payment Display

Test Details: View balances interface correctly illustrates pending payments specifically formatted matching database outputs on the graphical screen perfectly.

Test Owner: Ronit Kumar

Test Date: 29/03/2026

Test Results: Values mapped appropriately.



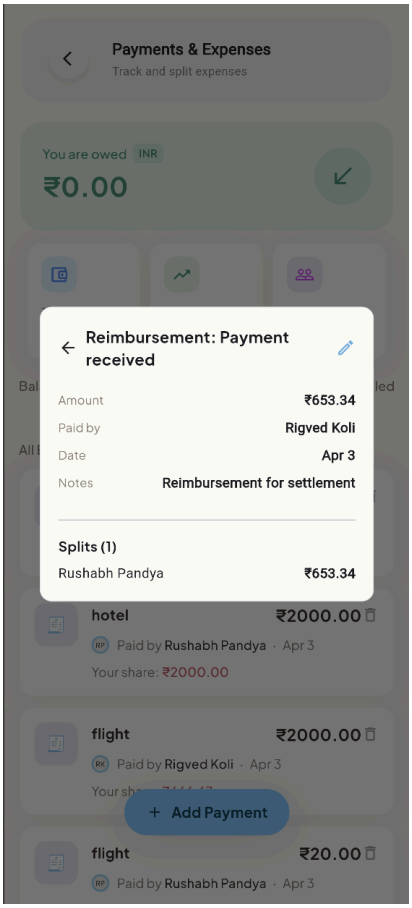
3.4.2 Settlement UI Reaction (Mark as Paid)

Test Details: Issuing a payment to the required recipient user successfully updates the system state, clearing the “you owe” balance for that user in the UI.

Test Owner: Ronit Kumar

Test Date: 29/03/2026

Test Results: “You owe” value cleared correctly after payment; UI updates accurately to reflect settled balance.



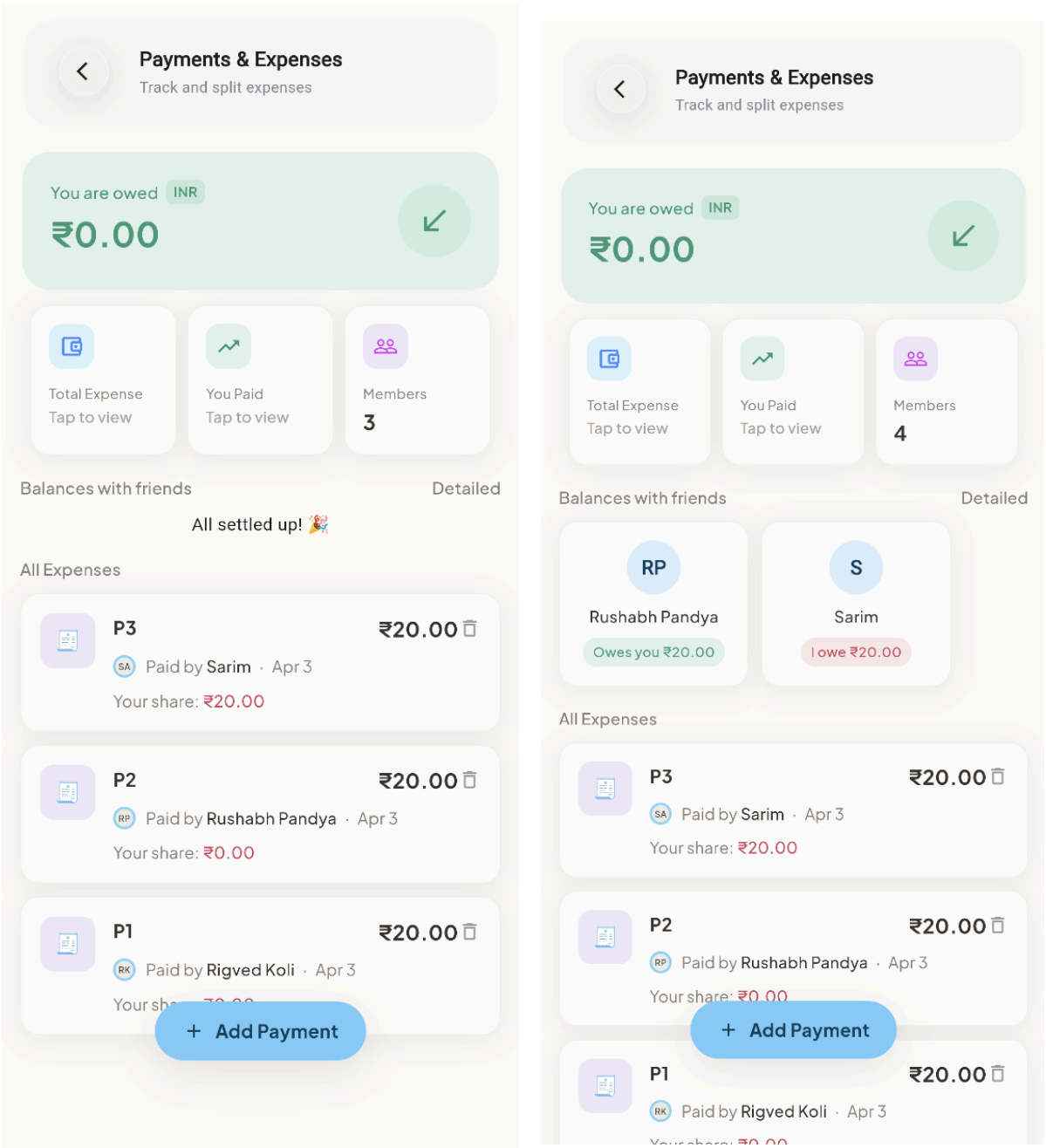
3.4.3 Simplify Debts

Test Details: Toggling the “Simplify Debts” option in settings dynamically alters the transaction visualization logic. When enabled, the system minimizes the number of transactions by simplifying balances; when disabled, it displays raw transaction paths. Test scenario executed: A pays B, B pays C, and C pays A (₹20 each). With Simplify Debts ON, circular debts are resolved and minimized; with it OFF, all individual transactions are shown explicitly.

Test Owner: Khushi

Test Date: 29/03/2026

Test Results: Toggle correctly switches between simplified and raw transaction views; circular debt reduced effectively when enabled.



3.5 Settlement ↔ External Flow

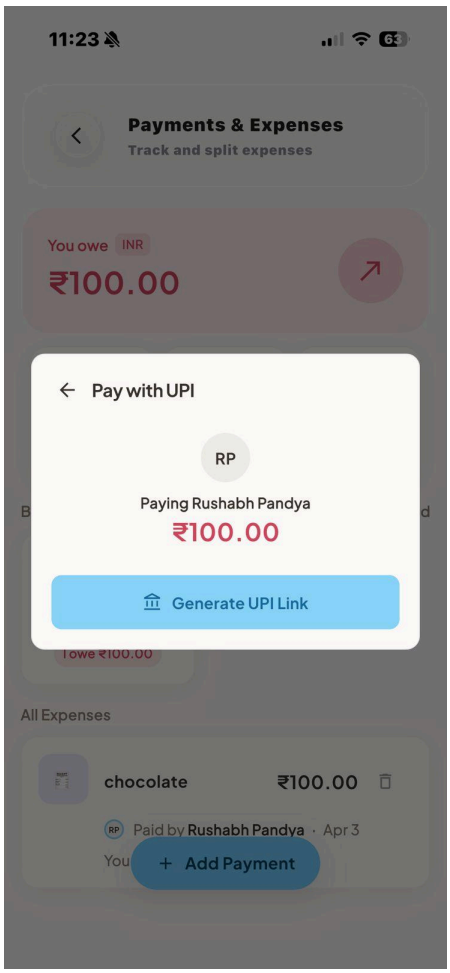
3.5.1 UPI Redirect

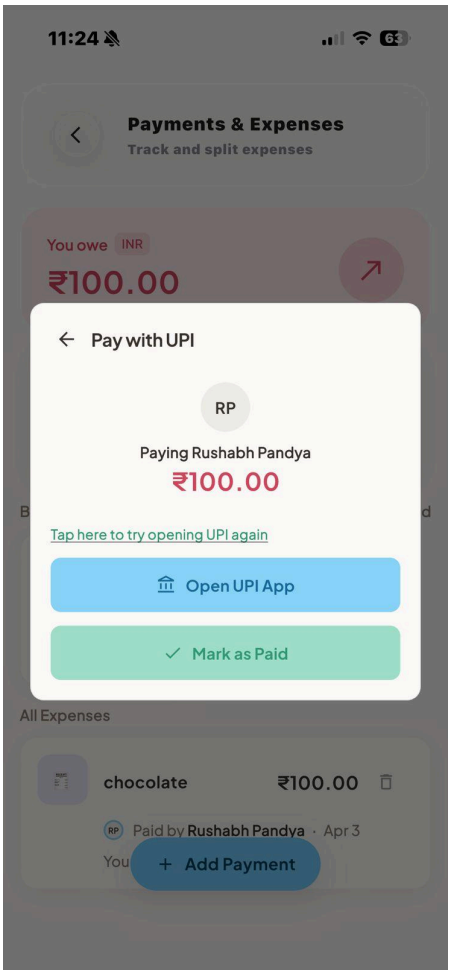
Test Details: Selecting “Pay with UPI” generates a valid UPI deep link. On clicking “Open UPI App,” the system redirects the user to their default UPI application on iOS, while on Android it presents a list of available UPI apps for the user to choose from.

Test Owner: Sanket

Test Date: 02/04/2026

Test Results: UPI link generation and deep linking function correctly







3.6 Route ↔ Trip

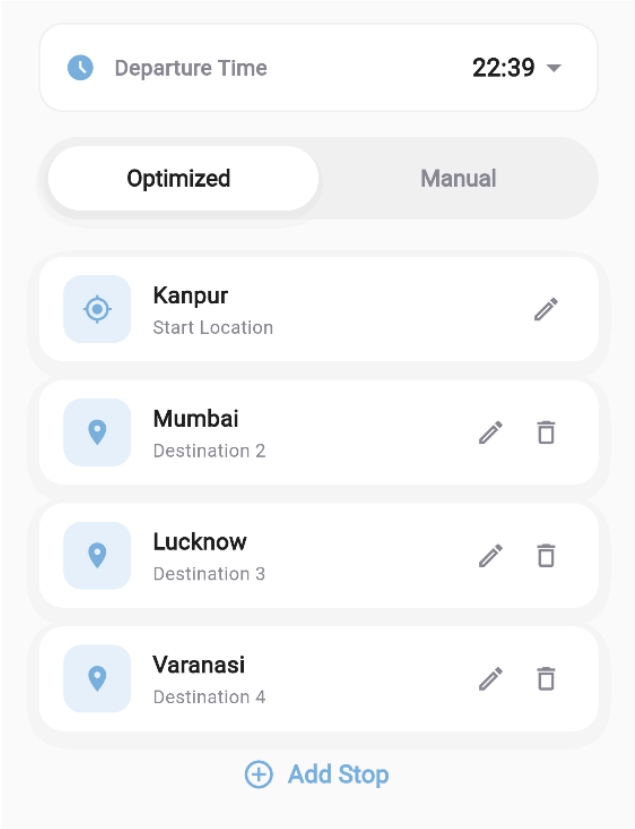
3.6.1 Route Planning

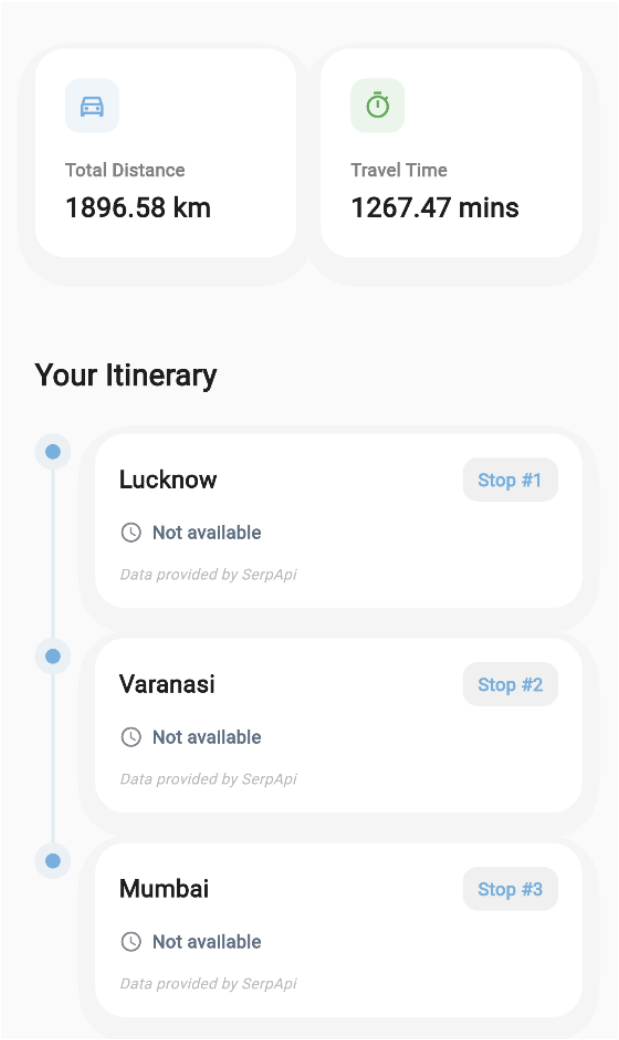
Test Details: Entering multiple places as stops triggers the backend route optimization service, which computes the shortest possible route connecting all stops and returns the optimized path to the frontend. The computed route is then displayed correctly on the user interface with updated ordering and distance.

Test Owner: Rushabh

Test Date: 3/4/26

Test Results: Route successfully optimized and displayed with correct stop order and shortest path.





4 System Testing

1. Requirement: Seamless User Authentication and Onboarding

The user opens the mobile application and begins the registration or login process via Firebase Auth. The application prompts them to complete their personal details or performs an asynchronous sync if signing in with Google. The onboarding checks against pre-existing emails gracefully.

Test Owner: Sarim
Test Date: 3/4/2026
Test Results: The test was conducted successfully. To test, the user opened the app, registered manually and then later through Google OAuth. Both operations concluded properly traversing loading mechanisms, correctly creating profiles linked across the Neon DB and Firebase Console without duplications.

2. Requirement: Comprehensive Trip Group Management

Any verified user can execute trip creations, defining variables such as destination, duration, place and specific trip labels. Moreover, user can add people in the trip, by entering their phone number.

Test Owner: Khushi
Test Date: 3/4/2026
Test Results: The test was conducted successfully. To test, the user filled the trip form completely. The resulting payload stored all string texts safely. Then, users natively transitioned into the app properly landing directly onto the group's specific dashboard.

3. Requirement: Ledger Accounting and Dynamic Expanse Splits

Any participant explicitly within the group confines should hold permissions to document financial obligations mapping variables like currency, date, and distribution strategies (Equal, Custom).

Test Owner: Raghav
Test Date: 3/4/2026
Test Results: The test was conducted successfully. Testing consisted of bombarding the ledger with diverse fractional numbers utilizing custom splits to attempt breakage points. It resulted smoothly without application crashes.

Additional Comments: Backend math validations mandate custom splits MUST sum to the identical value declared at the root payload stopping arbitrary numbers artificially mutating group net-money.

4. Requirement: Settlement Tracking and Simplification Algorithms

Clear indicators must highlight net gains or outstanding debts for every active user spanning the lifetime of the application, alongside dynamic pathing choices relying on debt simplification logics.

Test Owner: Sanket
Test Date: 3/4/2026

Test Results: The test was conducted successfully. The user navigated into the tracking ledger viewing graphical numbers mirroring NeonDB exactly without fail. They toggled simplification on, resolving heavy cyclic networks (A owes B, B owes C) into single transaction resolutions rapidly resulting locally on the visual UI.

Additional Comments: Simplification logic drastically reduces computational overhead.

5. Requirement: Document and Photos Tracking Repository

Users can also add/remove photos as well as documents related to trip.The user navigates to the trip dashboard and opens the shared repository. They can seamlessly upload photos from their phone gallery or attach important PDFs like travel tickets. The application makes them accessible to all group members, and gracefully allows users to securely delete any media or documents they previously added.

Test Owner: Pranjali
Test Date: 3/4/2026
Test Results:

6. Requirement: Itinerary Mapping

The user inputs multiple points of interest (POIs) and can choose between two routing modes: optimal or manual. In 'optimal mode', the application seamlessly processes the scattered locations and automatically calculates the most efficient travel route to minimize commuting time. Alternatively, in 'manual mode', the user has the flexibility to freely reorder the stops and create their own custom itinerary.

Test Owner: Ayush
Test Date: 3/4/2026
Test Results: The test was conducted successfully. The user provided several POIs (Points of Interest).

5 Conclusion

Testing Effectiveness and Exhaustiveness Testing was conducted comprehensively using the actual mobile frontend and manual calls using Postman, ensuring a robust and reliable user experience. Each user flow was rigorously evaluated end-to-end. This included verifying that Firebase authentication tokens correctly passed to the backend services, executed accurate modifications to the NeonDB database, and reflected the desired updated information seamlessly on the mobile screens. Furthermore, the validity of each input was thoroughly verified across all forms, explicitly handling complex calculations such as custom expense amounts and selective or equal payment splits.

Challenges Encountered During Testing The main difficulty faced during the testing phase was capturing the exact database state at the precise moment complex settlement algorithms—such as debt simplification—executed. Additionally, simulating real-time, multi-user group interactions natively on the mobile device proved challenging, as it required the use and management of multiple test accounts to properly replicate authentic group dynamics.

Future Improvements While the current manual testing strategy was exhaustive, the overall testing process could be significantly improved by incorporating automated End-to-End (E2E) testing frameworks tailored for mobile platforms, such as Appium or Detox. Implementing these automated tools would drastically reduce the manual overhead associated with traversing application screens and repetitively polling the NeonDB client, streamlining the validation process for future development cycles.

Appendix A - Group Log

Date	Timing	Duration	Work Done
March 28	21:00–22:30	1.5 hrs	• Identified key features to be tested
March 29	20:00–22:00	2 hrs	• Performed unit testing on core features (authentication, expense creation) • Verified database updates for each operation
March 31	20:00–22:00	2 hrs	• Performed unit testing on core features (photo/document upload, route planning) • Verified interaction between multiple features
April 2	22:00–23:00	1 hrs	• System testing of complete application • Tested end-to-end user workflows
April 3	21:00–00:00	3 hrs	• Final validation, bug fixing and readiness check